

Name: Pratikshya Chand

Class : _____ Subject : _____

[illegible]

Sets - A set is a collection of well defined objects. For example the collection of books in a library.

Sets are denoted by using capital letters and the elements of set are denoted by small letters.

Various ways of describing a set -

1. **By listing elements:** For example, the set of all positive integers less than 6 can be written as $\{1, 2, 3, 4, 5\}$
2. **By describing the properties of the elements -**
For example, the set $\{1, 2, 3, 4, 5\}$ can be described as : $\{n | n \text{ is a positive integer less than } 6\}$
3. **By recursion -** We define the elements of the set by a computational rule for calculating the elements. For example, the set of all natural numbers leaving a remainder 1 when divided by 3 can be described as $\{a_n | a_0 = 1 \text{ and } a_{n+1} = a_n + 3\}$
This gives $\{1, 4, 7, 10, \dots\}$

Types of sets -

1. **Empty set** - A set is said to be an empty set if it contains no elements. It is denoted by $\{\phi\}$
2. **Singleton set** - A singleton set contains only one element. eg - $A = \{1\}$

3. Finite set - A finite set contains fixed number of elements. It contains countable elements.
4. Infinite set - An infinite set has no finite number of elements or cannot be counted.
5. Disjoint set - Two sets are said to be disjoint set if they have no elements in common.
6. Overlapping set - Two sets are said to be overlapping set if they have at least one element in common.
7. Subset - Set A is said to be subset of set B if all the elements of A is present in B.

Some operations on set -

1. Union - The union of sets A and B, $A \cup B$, is a set which contains elements in A or B or both.
 $A \cup B = \{x : x \in A \text{ or } x \in B\}$
2. Intersection - The intersection of sets A and B, $A \cap B$, is a set which elements present in A and B both.
 $A \cap B = \{x : x \in A \text{ and } x \in B\}$
3. Set difference - $A - B = \{x : x \in A \text{ and } x \notin B\}$
4. Complement - The complement of set A is a set containing everything that is not in A.

Relations - A relation is a set of ordered pairs that show the relationship between two values. eg- $\{(1,2), (3,4)\}$

Properties of relation -

1. Reflexive property - A relation R in a set S is reflexive if xRx for every x in S .
2. Symmetric property - A relation R in a set S is symmetric if for x, y in S , yRx whenever xRy .
3. Transitive property - A relation R in a set S is transitive if for x, y, z in S , xRz whenever xRy and yRz .

Equivalence relation - A relation R in a set S is called equivalence relation if it's reflexive, symmetric and transitive.

Function - A function or map 'f' from X to Y is a rule that assigns each element x of X exactly with unique element of Y . The function is denoted by $f: X \rightarrow Y$.

Alphabets - An alphabet is a non-empty set of symbols ; or letters or digits. It is denoted by Σ . eg- $\Sigma = \{0, 1\}$ is the binary alphabet.

Strings or word - It is a finite sequence of symbols chosen from alphabet.

- * Σ = alphabet
- * Σ^1 = set of strings
- * $L = \{\emptyset\}$ - empty
- * $L = \{\epsilon\}$ - not empty

$$\Sigma^0 = \{\epsilon\}$$

$$\Sigma^k, \Sigma^L = \Sigma^*$$

DATE: ___/___/___
PAGE: _____

eg- $A = \{a, b\}$

String: $\{aa, ab, aba, \dots\}$

- * Empty string - String with zero occurrence of symbols. It is denoted by ϵ .
- * Length of string - It is the number of positions for symbols in the string. It is denoted by $w: |w|$. eg- $|111| = 3$, $|\epsilon| = 0$.

Concatenation of strings - If $w_1 = ab$ and $w_2 = bc$ are two strings, then the concatenation of strings is given by $w_1.w_2 = abbc$.

Closure of alphabets - It is the set of all strings including empty string. eg- $\Sigma = \{0, 1\}$
 $\Sigma^* = \{\epsilon, 0, 1, 01, 11, 101, \dots\}$

⇒ The symbol $*$ is called Kleene star.

$$\Sigma^+ = \{0, 1, 11, \dots\}$$

$$\text{Thus, } \Sigma^* = \Sigma^+ \cup \{\epsilon\}$$

Language - Language is the collection of all possible strings.

$$\Sigma = \{0, 1\} \quad L = \{0, 1, 01, 10, 010, 000, \dots\}$$

L = set of all string that begins with 0

$$\therefore L = \{0, 01, 001, 000, \dots\}$$

- * Σ^k where k is the length of the string.

$$\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \dots$$

$$L \subseteq \Sigma^*$$

Grammar - It is a rule or format system for accepting or rejecting string/language.

CHAPTER 2 FINITE AUTOMATA AND REGULAR EXPRESSION

Automata is a computational device that accepts the strings of a language L over an input alphabet Σ .

Finite state machine - A finite state machine is defined as the system of automation where energy, material and information are transformed and used for performing some functions without direct involvement of a man. eg- automatic machine tools, automatic packing machine, etc.

An automation in which the output depends only on input is called automation without a memory.

An automation in which the output depends on input as well as the states is called automation with finite memory.

An automation in which the output depends only on state of machine is called Moore machine.

An automation in which the output depends on

the input as well as on the states at any instant of time is called Mealy machine.

Finite State Automata (FSA) - Finite state automata are special kind of finite state machine with no outputs but have set of final states. They can recognize the string that take the starting state to final state but no storage mechanism so, sometimes called automation without memory.

- Can be used as language recognizers.
- Applied to design and construct compilers.
- * The set of strings recognized by finite automata are called regular expression.

Finite automata is a collection of 5-tuples $(Q, \Sigma, \delta, q_0, F)$ described as -

Q = finite non-empty set of states

Σ = finite non-empty set of input alphabets

δ = transition function, $Q \times \Sigma \rightarrow Q$

q_0 = initial state

F = set of final states (subset of Q)

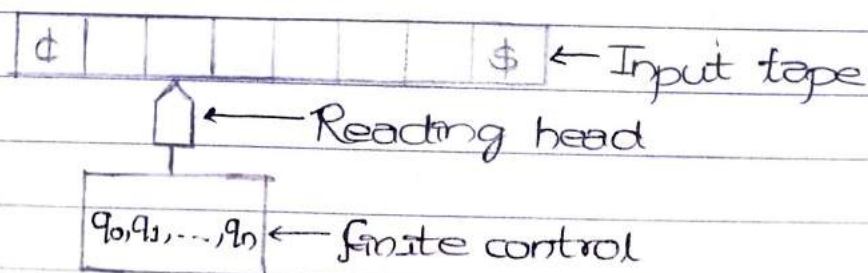
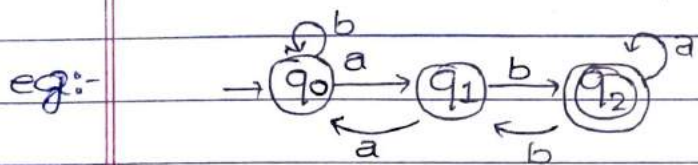


fig- finite automata

Types of Finite Automata -

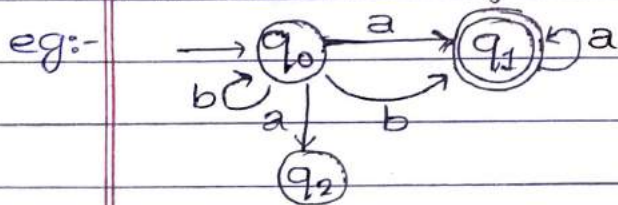
1. **Deterministic Finite Automata (DFA):** The finite automata is called DFA if there is only one path for a specific input from current state to next state. A DFA is a collection of five tuple $M = (Q, \Sigma, \delta, q_0, F)$ where
 - a) Q is a finite set of states
 - b) Σ is a finite set of input alphabets
 - c) δ = transition function
 - d) q_0 = initial state
 - e) F = set of final states



In the above DFA, for input 'a', at state q_0 , there is only one path going to q_1 and similar is the case for every other states.

2. **Non-deterministic Finite Automata (NFA):** The finite automata is called NFA if there exists many path for a specific input from current state to next state.

NFA can be defined as a collection of 5 tuples.



In the above NFA, for input 'a' at state q_0 , there are two next states q_1 and q_2 . Similarly, for input 'b' there are two next states q_0 and q_1 .

Q.1. Given string is 110101. Check whether the string is accepted or not. Transition function is given below:

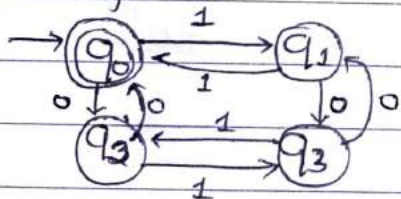
States	Input 0	Input 1
$\rightarrow q_0$	q_2	q_1
q_1	q_3	q_0
q_2	q_0	q_3
q_3	q_1	q_2

$\Rightarrow$ Given string is 110101.

Initial and final state = q_0

Now, $(q_0, 110101) \vdash \delta(q_1, 10101)$
 $\vdash \delta(q_0, 0101)$
 $\vdash \delta(q_2, 101)$
 $\vdash \delta(q_3, 01)$
 $\vdash \delta(q_1, 1)$
 $\vdash \delta(q_0, \phi)$

Since q_0 is the final state, the given string is accepted.

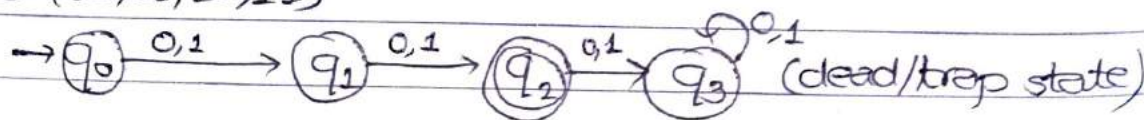


2. Design DFA : (i) that accepts the start of string over $\Sigma = \{0, 1\}$ of length 2.

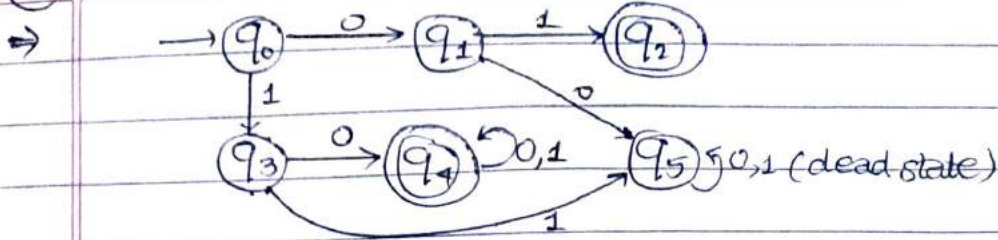
Sol: Given, length = 2, $\Sigma = \{0, 1\}$

Possible no. of states = 3

$L = \{00, 01, 10, 11\}$

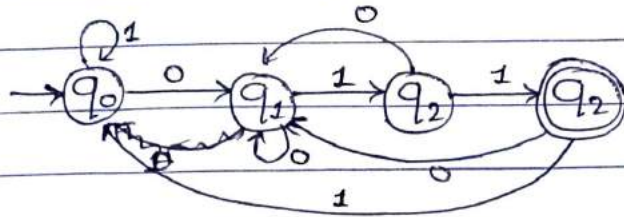


ii) that accepts 01 or a string of atleast '1' followed by 0.



iii) that accepts string that ends with 011 over $\Sigma = \{0,1\}$

sol:



3. Consider the finite automata in which

$Q = \{q_0, q_1, q_2, q_3\}$, $\Sigma = \{0,1\}$, $q_0 = \{q_0\}$, $F = \{q_0\}$

δ is given by

State	Input	
	0	1
q_0	q_1	q_1
q_1	q_3	q_0
q_2	q_0	q_3
q_3	q_1	q_2

1) Find whether the given string is accepted or not:

a) 11111 b) 101101

a) Given string is 11111.

$(q_0, 11111) \vdash \delta(q_1, 11111)$

$\vdash \delta(q_0, 1111)$

$\vdash \delta(q_1, 11)$

$\vdash \delta(q_0, 1)$

$\vdash \delta(q_1, \phi)$

Since, q_1 is not final state, the given string is not accepted.

b) Given string is 101101.

$$\begin{aligned}
 (q_0, 101101) &\vdash \delta(q_1, 01101) \\
 &\vdash \delta(q_3, 1101) \\
 &\vdash \delta(q_2, 101) \\
 &\vdash \delta(q_3, 01) \\
 &\vdash \delta(q_1, 1) \\
 &\vdash \delta(q_0, \phi)
 \end{aligned}$$

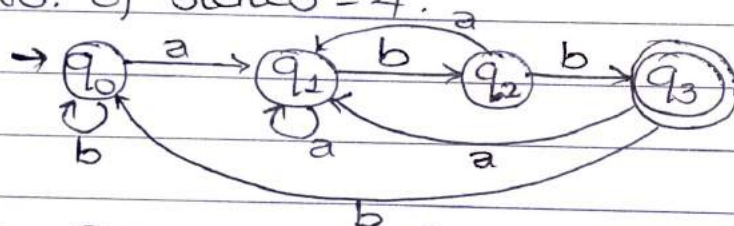
Since, q_0 is the final state, the given string is accepted.

Sequence of states: $q_0, q_1, q_3, q_2, q_3, q_1, q_0$.

4. Design DFA to accept string of a's and b's ending with abb over $\Sigma = \{a, b\}$.

Length of string = $|abb| = 3$

No. of states = 4.



So, DFA is $M = \{Q, \Sigma, \delta, q_0, F\}$ where

$$Q = \{q_0, q_1, q_2, q_3\}$$

$$\Sigma = \{a, b\}$$

$$q_0 = \{q_0\}$$

$$F = \{q_3\}$$

δ is given by:

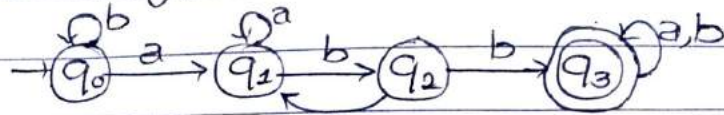
States	Input	
	a	b
q_0	q_1	q_0
q_1	q_1	q_2
q_2	q_1	q_3
q_3	q_1	q_0

Suppose the string is bbabb.

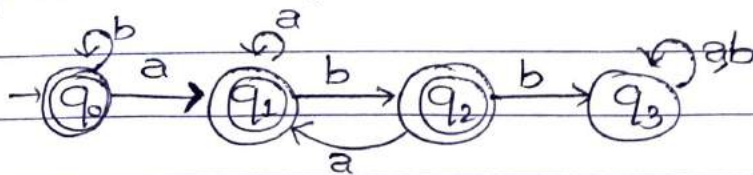
$$\begin{aligned}
 \text{Then, } (q_0, bbabb) &\vdash (q_0, babb) \\
 &\vdash (q_0, abb) \\
 &\vdash (q_1, bb) \\
 &\vdash (q_2, b) \\
 &\vdash (q_3, \phi)
 \end{aligned}$$

5. Design a DFA to accept any string of a's and b's not containing 'abb' substring over $\Sigma = \{a, b\}$.

⇒ Initially, we make DFA that accepts substring 'abb'.

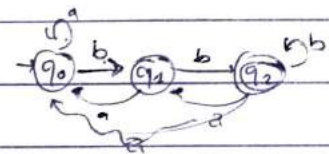
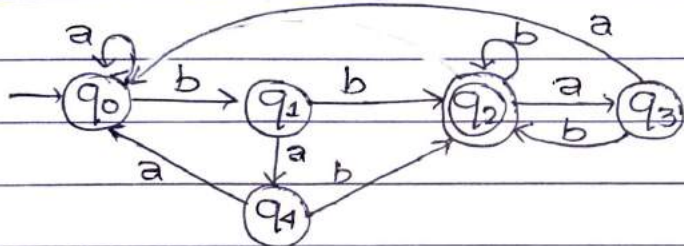


Now, for not containing 'abb' substring, we change final to non-final state and vice-versa.



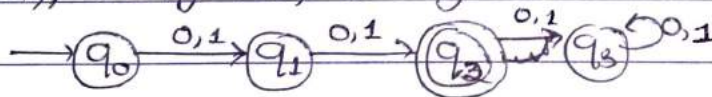
6. Design a DFA to accept string ending with bb or bab over $\Sigma = \{a, b\}$.

⇒



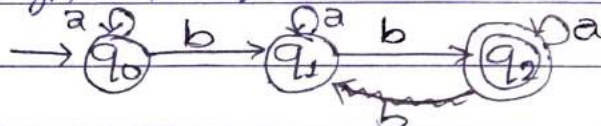
7. Construct NFA for set of all string over $\Sigma = \{0, 1\}$ of length 2.

⇒ Given, Length of string = 2



8. Design DFA that checks even no. of b's.

⇒



Equivalence of NFA and DFA:

For every NFA, there is DFA.

For NFA, $M = \{Q, \Sigma, \delta, q_0, F\}$

For DFA, $M' = \{Q', \Sigma, \delta', q_0, F'\}$ where

$Q' = 2^Q$, $\Sigma = \Sigma$, $q_0' = q_0$

$F' =$ set of all states that contain final state of NFA. (set of subset containing all elements of F and subset of $Q = \{q_0\}$ & $\{q_0q_1\}$)

* $M = (\{q_0, q_1\}, \{0, 1\}, \delta, \{q_0\}, \{q_1\})$ and state table i.e δ is given as:

⇒ For DFA,

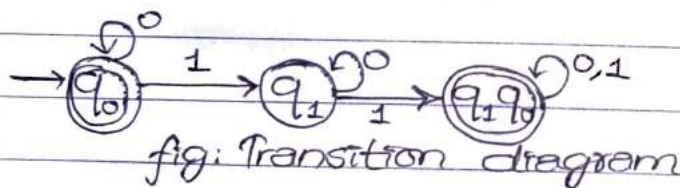
* $M' = \{Q', \Sigma, \delta', q_0', F'\}$

where $Q' = 2^Q = 2^2 = 4$

$\Sigma = \{0, 1\}$, $q_0' = q_0$, $F = \{q_0, q_0q_1\}$

δ' is given as:

State	Input	
	0	1
q_0	q_0	q_1
q_1	q_1	$[q_0q_1]$
$[q_0q_1]$	$[q_0q_1]$	$[q_1q_0]$



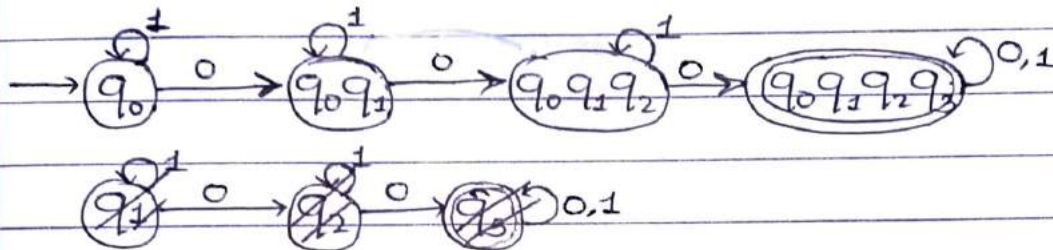
1. Construct DFA equivalent to NFA, $M = (\{q_0, q_1, q_2, q_3\}, \{0, 1\}, \{q_0\}, \{q_3\})$.

State	Input	
	0	1
q_0	q_0q_1	q_0
q_1	q_2	q_1
q_2	q_3	q_2
q_3	q_3	q_3

⇒ For DFA, $M' = \{Q', \Sigma, \delta', q_0, F'\}$ where
 $Q' = 2^Q = 2^4 = 16$, $\Sigma = \{0, 1\}$, $q_0' = q_0$.

States	Input 0	1
q_0	$[q_0 q_1]$	q_0
q_1	q_2	q_1
q_2	q_3	q_1
q_3	q_3	q_3

$[q_0 q_1]$ $[q_0 q_1 q_2]$ $[q_0 q_1]$
 $[q_0 q_1 q_2]$ $[q_0 q_1 q_2 q_3]$ $[q_0 q_1 q_2]$ $F' = \{[q_0 q_1 q_2 q_3]\}$
 $[q_0 q_1 q_2 q_3]$ $[q_0 q_1 q_2 q_3]$ $[q_0 q_1 q_2 q_3]$ $Q' = \{q_0, [q_0 q_1], [q_0 q_1 q_2], [q_0 q_1 q_2 q_3]\}$



2. $M = (\{q_1, q_2, q_3\}, \{0, 1\}, \delta, \{q_1\}, \{q_3\})$

Construct a DFA from above NFA if δ is given by

$$\delta(q_1, 0) = \{q_2 q_3\}$$

$$\delta(q_1, 1) = \{q_1\} \quad \delta(q_2, 0) = \{q_1 q_2\} \quad \delta(q_2, 1) = \{\phi\}$$

$$\delta(q_3, 0) = \{q_2\} \quad \delta(q_3, 1) = \{q_1 q_2\}$$

⇒ For DFA, $M' = \{Q', \Sigma, \delta', q_0', F'\}$ where

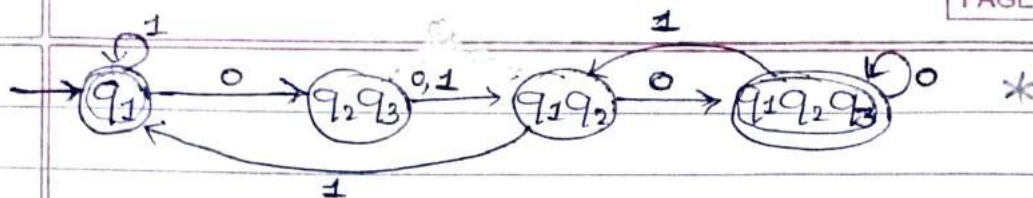
$$Q' = 2^Q = 2^3 = 8.$$

$$\Sigma = \{0, 1\}$$

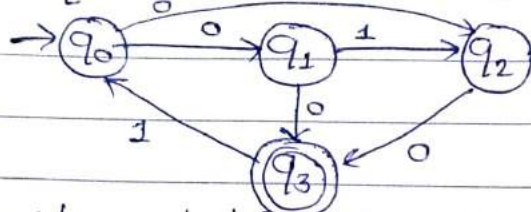
$$q_0' = \{q_1\} \quad F' = \{q_1, [q_1 q_2], [q_1 q_2 q_3]\}$$

δ' is given as: States Input

q_1	$[q_2 q_3]$	q_1
q_2	$[q_1 q_2]$	ϕ
q_3	q_2	$[q_1 q_2]$
$[q_1 q_2]$	$[q_1 q_2 q_3]$	q_1
$[q_1 q_2]$	$[q_1 q_2]$	$[q_1 q_2]$
$[q_1 q_2 q_3]$	$[q_1 q_2 q_3]$	$[q_1 q_2]$



3. The transition diagram of NFA is given. Construct the equivalent DFA.



⇒ Transition table of NFA is :

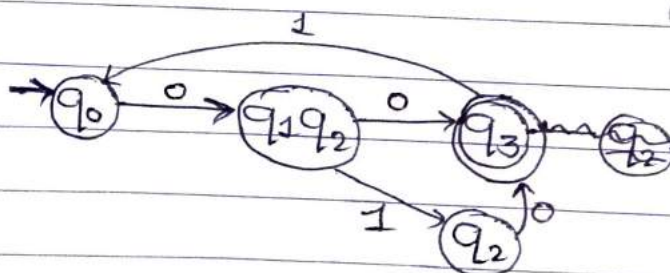
States	Inputs	
	0	1
q_0	q_1	ϕ
q_1	q_3	q_2
q_2	q_3	ϕ
q_3	ϕ	q_0

For DFA, $M' = \{Q', \Sigma, \delta', q_0', F'\}$ where,
 $Q' = 2^Q = 2^4 = 16$, $\Sigma = \{0, 1\}$, $q_0' = \{q_0\}$
 δ' is given by:

States	Inputs	
	0	1
q_0	$[q_1, q_2]$	ϕ
$[q_1, q_2]$	q_3	q_2
q_2	q_3	ϕ
q_3	ϕ	q_0

$$\therefore F' = \{q_3\}$$

$$Q' = \{q_0, q_2, q_3, [q_1, q_2]\}$$



4. $M = \{q_1, q_2, q_3\}, \{0, 1\}, \delta, \{q_1\}, \{q_3\}$ is NFA given by

$$\delta(q_1, 0) = \{q_2, q_3\}$$

$$\delta(q_2, 1) = \{\phi\}$$

$$\delta(q_1, 1) = \{q_1\}$$

$$\delta(q_3, 0) = \{q_2\}$$

$$\delta(q_2, 0) = \{q_1, q_2\}$$

$$\delta(q_3, 1) = \{q_1, q_2\}$$

Construct equivalent DFA.

⇒ For NFA : States Inputs

$$q_1 \quad q_2, q_3 \quad q_1$$

$$q_2 \quad q_1, q_2 \quad \phi$$

$$q_3 \quad q_2 \quad q_1, q_2$$

For DFA, δ' is given by :

States Inputs

$$q_1 \quad [q_2, q_3] \quad q_1$$

$$[q_2, q_3] \quad [q_1, q_2] \quad [q_1, q_2]$$

$$[q_3, q_2] \quad [q_1, q_2, q_3] \quad [q_1]$$

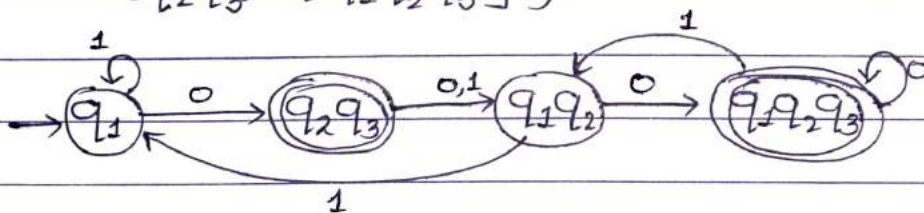
$$[q_1, q_2, q_3] \quad [q_1, q_2, q_3] \quad [q_1, q_2]$$

∴ The equivalent DFA is $M' = \{\Phi', \Sigma, \delta', q_0^*, F'\}$ where

$$\Phi' = \{q_1, [q_2, q_3], [q_1, q_2], [q_1, q_2, q_3]\}$$

$$\Sigma = \{0, 1\}, \quad q_0^* = \{q_1\}$$

$$F' = \{[q_2, q_3], [q_1, q_2, q_3]\}$$



5. Construct DFA equivalent to NFA $M = \{q_0, q_1, q_2, q_3\}, \{0, 1\}, \delta, \{q_0\}, \{q_3\}$

State Input

$$q_0 \quad [q_0, q_1] \quad q_0$$

$$q_1 \quad q_2 \quad q_1$$

$$q_2 \quad q_3 \quad q_2$$

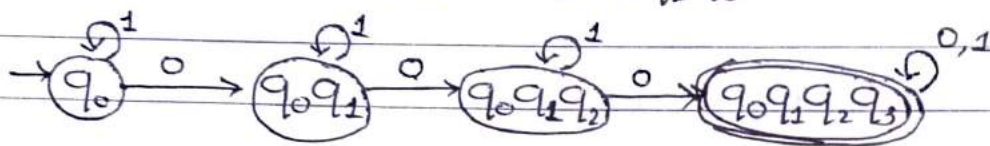
$$q_3 \quad q_3 \quad q_3$$

⇒ For ZFA, $M' = \{Q', \Sigma, \delta', q_0', F'\}$ where
 $Q' = 2^Q = 2^4 = 16$, $\Sigma = \{0, 1\}$, $q_0' = \{q_0\}$

δ' is given by:

States Inputs

q_0	$[q_0 q_1]$	q_0
$[q_0 q_1]$	$[q_0 q_1 q_2]$	$[q_0 q_1]$
$[q_0 q_1 q_2]$	$[q_0 q_1 q_2 q_3]$	$[q_0 q_1 q_2]$
$[q_0 q_1 q_2 q_3]$	$[q_0 q_1 q_2 q_3]$	$[q_0 q_1 q_2 q_3]$



∴ $Q' = \{q_0, [q_0 q_1], [q_0 q_1 q_2], [q_0 q_1 q_2 q_3]\}$
 $F' = \{[q_0 q_1 q_2 q_3]\}$

Regular expression - Regular expression represents certain set of strings in algebraic fashion. It describes the language accepted by finite automata (RL).

It can be defined as:

1. Every element of alphabet (Σ) is a regular expression.
2. Null string (ϵ) or empty string (ϕ) are also r.e.
3. The Union of two regular expressions R_1 and R_2 i.e. $(R_1 + R_2 / R_1, R_2)$ is also a r.e.
4. The concatenation of two regular expressions, R_1 and R_2 i.e. $R_1 R_2$ is also a r.e.
5. The iteration (closure) of a regular expression $R^{(R^*)}$ is also a r.e.

1. Write r.e for following:

- a) ~~Write~~ $\{101\} = 101$
 b) $\{01, 10\} = 01+10$
 c) For all the combination of a's except null string.
 $\Rightarrow \{a^+\}$
 d) For all the combination of a's over $\Sigma = \{a\} \Rightarrow a^*$
 e) For all the strings containing any no. of a's & b's.
 $\Rightarrow (a+b)^*$
 f) For all the strings that start with 1 and end with 0 over $\Sigma = \{0, 1\}$. $\Rightarrow 1(0+1)^*0$
 g) For accepting all string starting and ending with a and having any combination of b.
 $\Rightarrow ab^*a$
 h) For string having length exactly 2 over $\Sigma = \{a, b\}$
 $\Rightarrow L = \{aa, ab, ba, bb\}$
 $\therefore$ ~~the~~ Union $= aa+ab+ba+bb = (a+b)(a+b)$
 $\therefore$ r.e is $(a+b)(a+b)$
 * Atleast two: $(a+b)(a+b)(a+b)^*$
 i) For string having even length over $\Sigma = \{a\}$
 $\Rightarrow (aa)^*$

Identities of regular expression:

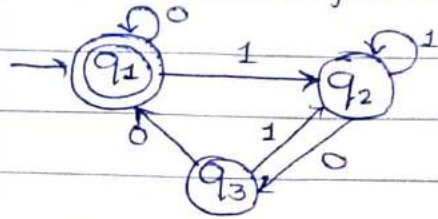
- | | |
|-------------------------------|--|
| 1. $\phi + R = R$ | 8. $(PQ)^*P = P(QP)^*$ |
| 2. $\phi R + R\phi = \phi$ | 9. $(P+Q)R = PR + QR$ |
| 3. $ER = RE = R$ | 10. $E+RR^* = E+R^*R = R^*$ |
| 4. $E^* = E$ and $\phi^* = E$ | 11. $(P+Q)^* = (P^*Q^*)^* = (P^*+Q^*)^*$ |
| 5. $R+R = R$ | |
| 6. $R^*R^* = R^*$ | |
| 7. $(R^*)^* = R^*$ | |

Conversion of DFA to r.e:

Arden's Theorem -

If P, Q, R are r.e and $R = Q + RP$. Then, $R = QP^*$

1. Find the r.e from following state diagram:



(incoming edges)

$$\Rightarrow q_1 = q_1 0 + q_3 0 + \epsilon \text{ (for initial state)} \dots (1)$$

$$q_2 = q_2 1 + q_3 1 + q_1 1 \dots (2)$$

$$q_3 = q_2 0 \dots (3)$$

From Substituting (3) in (2), we get,

$$q_2 = q_2 1 + q_2 0 1 + q_1 1$$

$$\Rightarrow \frac{q_2}{R} = \frac{q_2}{R} \frac{(1+01)}{P} + \frac{q_1 1}{Q}$$

Using Arden's theorem, $R = QP^*$, we get,

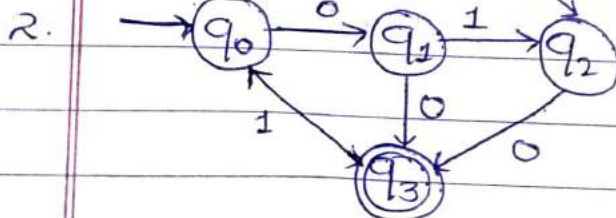
$$q_2 = q_1 1 (1+01)^*$$

$$\text{From (1), } q_1 = q_1 0 + q_2 0 0 + \epsilon$$

$$\Rightarrow \frac{q_1}{R} = \frac{q_1}{R} \frac{0}{Q} + \frac{q_2 1 (1+01)^* 0 0}{R \cdot P} + \frac{\epsilon}{Q}$$

$$\Rightarrow \frac{q_1}{R} = \frac{q_1}{R} \frac{(0 + 1(1+01)^* 0 0)}{P} + \frac{\epsilon}{Q}$$

Using Arden's theorem, $q_1 = \epsilon (0 + 1(1+01)^* 0 0)^*$
 $\therefore q_1 = (0 + 1(1+01)^* 0 0)^*$



$$\Rightarrow q_0 = q_3 1 + \epsilon \dots (1)$$

$$q_1 = q_0 0 \dots (2)$$

$$q_2 = q_1 1 + q_0 0 \dots (3)$$

$$q_3 = q_1 0 + q_2 0 \dots (4)$$

Substituting (2) in (3), $q_2 = q_0 0 1 + q_0 0$

$$\Rightarrow q_2 = q_0 (0 + 0 1)$$

$$\therefore q_2 = (q_3 1 + \epsilon) (0 + 0 1) \dots (5)$$

$$\text{Now, from (4), } q_3 = q_0 0 0 + (q_3 1 + \epsilon) (0 + 0 1) 0$$

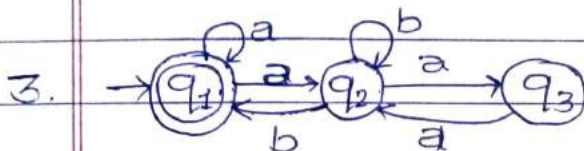
$$= (q_3 1 + \epsilon) 0 0 + (q_3 1 + \epsilon) (0 + 0 1) 0$$

$$\Rightarrow q_3 = (q_3 1 + \epsilon) (0 0 + 0 (0 + 0 1))$$

$$\therefore \frac{q_3}{R} = \frac{q_3 1}{R} \frac{(0 0 + 0 (0 + 0 1))}{P} + \frac{(0 0 + 0 (0 + 0 1))}{Q}$$

Using Arden's theorem, $q_3 = (0 0 + 0 (0 + 0 1))^* (1 (0 0 + 0 (0 + 0 1)))^*$

$$q_3 = (0 0 + 0 (0 + 0 1))^* (1 (0 0 + 0 (0 + 0 1)))^*$$



Construct regular expression from above state diagram.

$$\Rightarrow q_1 = q_1 a + q_2 b + \epsilon \dots (1)$$

$$q_2 = q_2 b + q_3 a + q_1 a \dots (2)$$

$$q_3 = q_2 a \dots (3)$$

From (2) and (3), $q_2 = q_2 b + q_2 a a + q_1 a$

$$\Rightarrow \frac{q_2}{R} = \frac{q_2}{R} \frac{(b + aa)}{P} + \frac{q_1 a}{Q}$$

Using Arden's theorem, $q_2 = q_1 a (b + aa)^*$

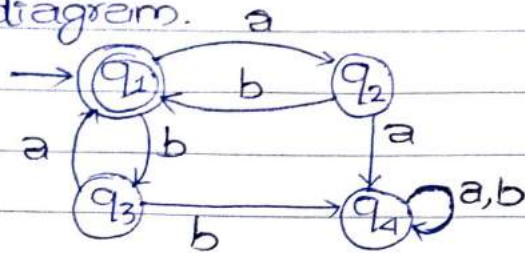
From (1), $q_1 = q_1 a + q_1 a (b + aa)^* b + \epsilon$

$$\Rightarrow \underline{q_1} = \underline{q_1 (a + a (b + aa)^* b)} + \underline{\epsilon}$$

Using Arden's theorem, $q_1 = \epsilon (a + a (b + aa)^* b)^*$

$$\therefore q_1 = (a + a (b + aa)^* b)^*$$

4. Construct regular expression from given state diagram.



$$\Rightarrow q_1 = q_2b + q_3a + \epsilon \quad \dots(1)$$

$$q_2 = q_1a \quad \dots(2)$$

$$q_3 = q_1b \quad \dots(3)$$

$$q_4 = q_2a + q_3b + q_4(a+b) \quad \dots(4)$$

$$q_4 =$$

$$\text{From (4), } q_4 = q_1aa + q_1bb + q_4(a+b)$$

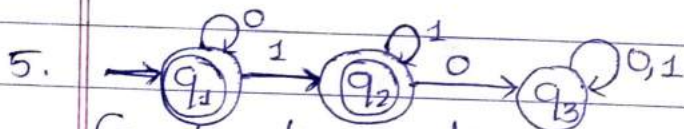
$$\Rightarrow q_4 = q_1(aa+bb) + q_4(a+b)$$

$$\text{From (1), } q_1 = q_1ab + q_1ba + \epsilon$$

$$\Rightarrow q_1 = q_1(ab+ba) + \epsilon$$

$$\text{Using Arden's theorem, } q_1 = \epsilon(ab+ba)^*$$

$$\therefore q_1 = (ab+ba)^*$$



Construct regular expression from state diagram given above.

$$\Rightarrow q_1 = q_10 + \epsilon \quad \dots(1)$$

$$q_2 = q_21 + q_11 \quad \dots(2)$$

$$q_3 = q_20 + q_3(0+1) \quad \dots(3)$$

$$\text{From (1), } q_1 = q_10 + \epsilon$$

$$\therefore q_1 = \epsilon 0^* = 0^* \quad (\text{Using Arden's theorem})$$

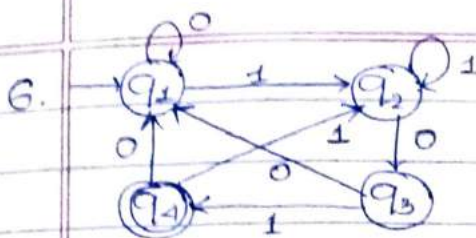
$$\text{From (2), } q_2 = q_21 + (q_10 + \epsilon)10^*$$

$$\text{Using Arden's theorem, } q_2 = (10^*)^*1$$

$$\text{So, the r.e is } q_1 + q_2 = 0^* + (10^*)^*1$$

$$= 0^*(\epsilon + 11^*) \quad (\because \epsilon + RR^* = R^*)$$

$$\therefore q_1 + q_2 = 0^*1^*$$



Construct regular expression.

$$\Rightarrow q_1 = q_1 0 + q_3 0 + q_4 0 + \epsilon \quad \dots (1)$$

$$q_2 = q_2 1 + q_1 1 + q_4 1 \quad \dots (2)$$

$$q_3 = q_2 0 \quad \dots (3)$$

$$q_4 = q_3 1 \quad \dots (4)$$

From (2) and (3), $q_2 = q_2 1 + q_1 1 + q_2 0 1$

$$\Rightarrow q_2 = q_2 (1 + 0 1 1) + q_1 1$$

Using Arden's theorem, $q_2 = q_1 1 (1 + 0 1 1)^*$

From (1), $q_1 = q_1 0 + q_2 0 + q_4 0 + \epsilon$

$$= q_1 0 + q_1 1 (1 + 0 1 1)^* (0 0 + 0 1 0) + \epsilon$$

$$= q_1 (0 + 1 (1 + 0 1 1)^* (0 0 + 0 1 0)) + \epsilon$$

$$\therefore q_1 = q_1 (0 + 1 (1 + 0 1 1)^* (0 0 + 0 1 0)) + \epsilon$$

Using Arden's theorem, $q_1 = (0 + 1 (1 + 0 1 1)^* (0 0 + 0 1 0))^*$

From (4), $q_4 = q_2 0 1$

$$= q_1 1 (1 + 0 1 1)^*$$

$$\therefore q_4 = (0 + 1 (1 + 0 1 1)^* (0 0 + 0 1 0))^* 1 (1 + 0 1 1)^*$$

(NFA with ϵ to NFA without ϵ)

Elimination of null move (ϵ/λ -move):

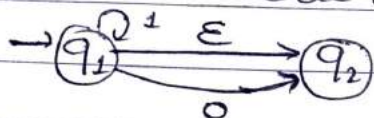
DATE: ___/___/___

PAGE: ___

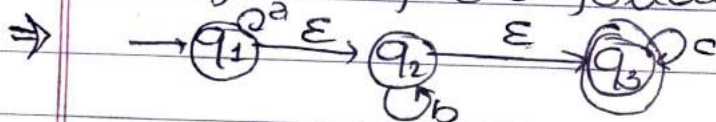
Suppose, we replace a null move from a vector v_1 and v_2 , then we proceed as follows:

1. Find all the edges starting from v_2 .
2. Duplicate all the edges from v_1 without changing edge label (symbol).
3. If v_1 is initial state, make v_2 as initial state.
4. If v_2 is final state, make v_1 as final state.

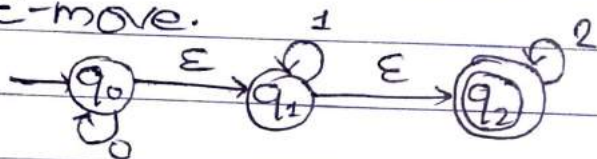
NFA with ϵ or λ move (null move): to change the state without any inputs



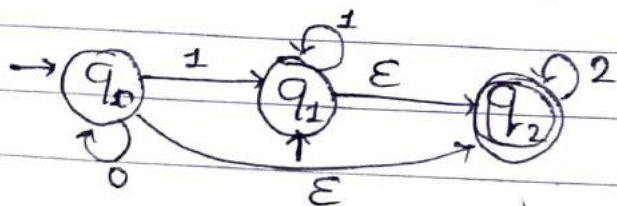
Construct NFA with ϵ for accepting strings having no. of a's followed by no. of b's or c's.

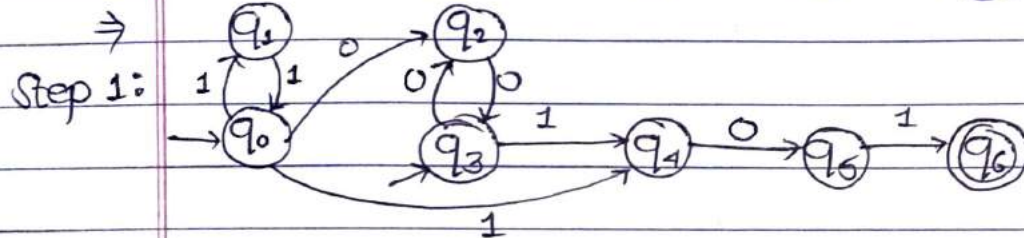
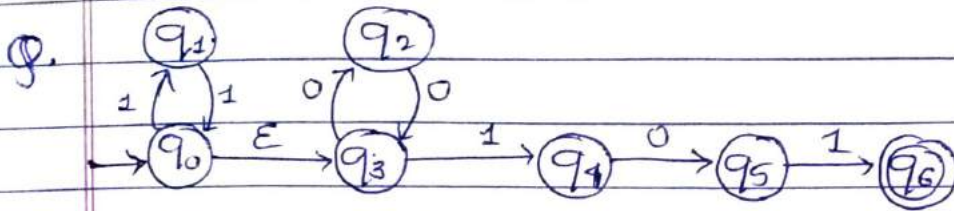
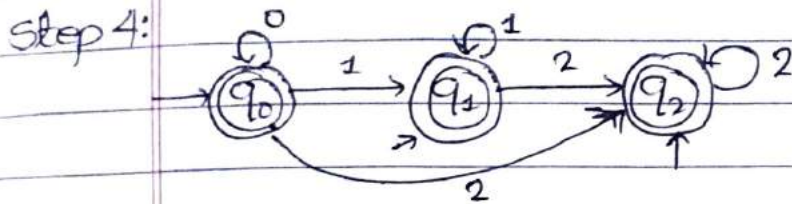
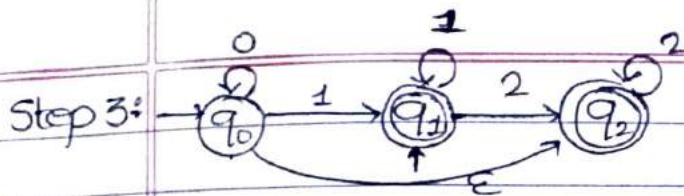


Q. Consider a FA with ϵ -move given below & construct an equivalent automata without ϵ -move.



$\Rightarrow$ Step 1: Consider q_0 and q_1 .
Step 2: Duplicate edges of v_2 from v_1 .

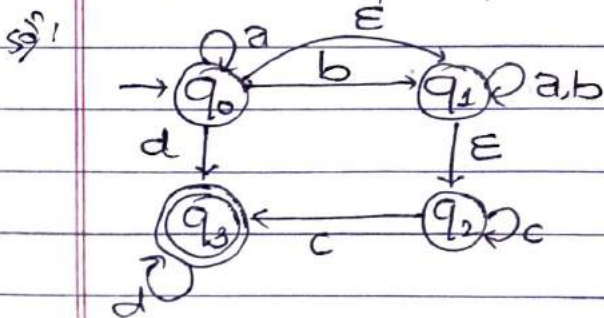




Q. Reduce the given NFA with ϵ to without ϵ if $M = \{\emptyset, \{a, b, c, d\}, \delta, q_0, \{q_3\}\}$.

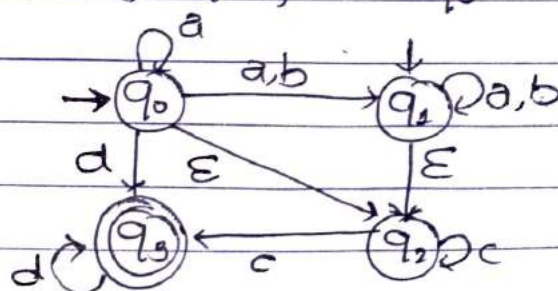
State table:

State	Input				
	a	b	c	d	ϵ
q_0	q_0	q_1	ϕ	q_3	q_1
q_1	q_1	q_1	ϕ	ϕ	q_2
q_2	ϕ	ϕ	$[q_2, q_3]$	ϕ	ϕ
q_3	ϕ	ϕ	ϕ	q_3	ϕ



Step 1: Consider q_0 & q_1 .

Step 2: Duplicate all edges of q_1 from q_0 .

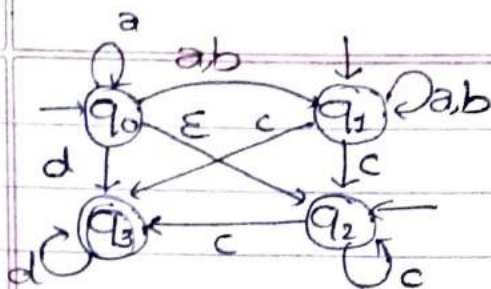


$\epsilon\text{-closure}(\phi) = \phi$

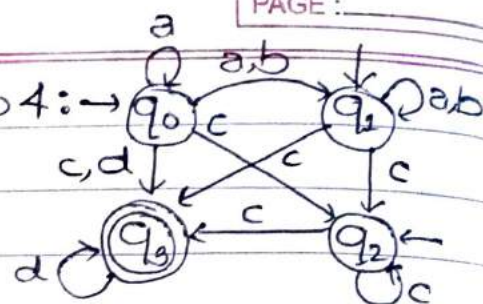
* states reachable with 1 move including current state

DATE: ___/___/___
PAGE: ___

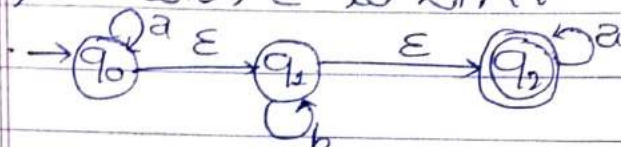
Step 3:



Step 4:



NFA with ϵ to DFA:



$$\Rightarrow \epsilon\text{-closure}(q_0) = \{q_0, q_1, q_2\}$$

$$\epsilon\text{-closure}(q_1) = \{q_1, q_2\}$$

$$\epsilon\text{-closure}(q_2) = \{q_2\}$$

We define δ' for DFA as:

$$\begin{aligned} a) \delta'(\{q_0, q_1, q_2\}, a) &= \epsilon\text{-closure}(\delta(\{q_0, q_1, q_2\}, a)) \\ &= \epsilon\text{-closure}(\delta(q_0, a), \delta(q_1, a), \delta(q_2, a)) \\ &= \epsilon\text{-closure}(q_0 \cup \phi \cup q_1) \\ &= \epsilon\text{-closure}(q_0) \cup \epsilon\text{-closure}(q_1) \\ &= \{q_0, q_1, q_2\} \cup \{q_2\} \\ &= \{q_0, q_1, q_2\} \end{aligned}$$

$$\begin{aligned} \delta'(\{q_0, q_1, q_2\}, b) &= \epsilon\text{-closure}(\delta(\{q_0, q_1, q_2\}, b)) \\ &= \epsilon\text{-closure}(\delta(q_0, b) \cup \delta(q_1, b) \cup \delta(q_2, b)) \\ &= \epsilon\text{-closure}(\phi \cup q_1 \cup \phi) \\ &= \{q_1, q_2\} \rightarrow \text{New state} \end{aligned}$$

$$\begin{aligned} b) \delta'(\{q_1, q_2\}, a) &= \epsilon\text{-closure}(\delta(\{q_1, q_2\}, a)) \\ &= \epsilon\text{-closure}(\delta(q_1, a) \cup \delta(q_2, a)) \\ &= \epsilon\text{-closure}(\phi \cup q_2) \\ &= q_2 \rightarrow \text{New state} \end{aligned}$$

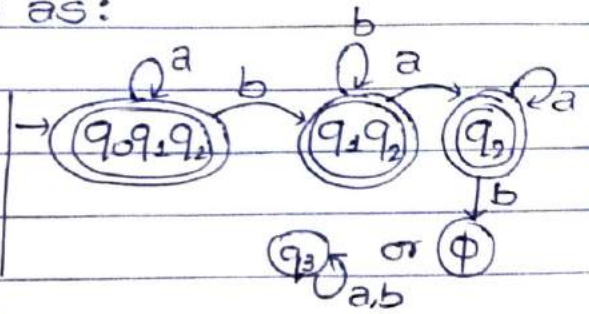
$$\begin{aligned} \delta'(\{q_1, q_2\}, b) &= \epsilon\text{-closure}(\delta(\{q_1, q_2\}, b)) \\ &= \epsilon\text{-closure}(q_1 \cup \phi) \\ &= \{q_1, q_2\} \end{aligned}$$

c) $\delta'(\{q_2\}, a) = \epsilon\text{-closure}(\delta(q_2), a)$
 $= \epsilon\text{-closure}(\{q_1, a\})$
 $= \{q_2\}$

$\delta'(\{q_2\}, b) = \epsilon\text{-closure}(\delta(q_2), b) = \phi$

Now, state table for DFA is as:

States	Input a	Input b
$\{q_0, q_1, q_2\}$	$\{q_0, q_1, q_2\}$	$\{q_1, q_2\}$
$\{q_1, q_2\}$	$\{q_2\}$	$\{q_1, q_2\}$
$\{q_2\}$	$\{q_2\}$	ϕ

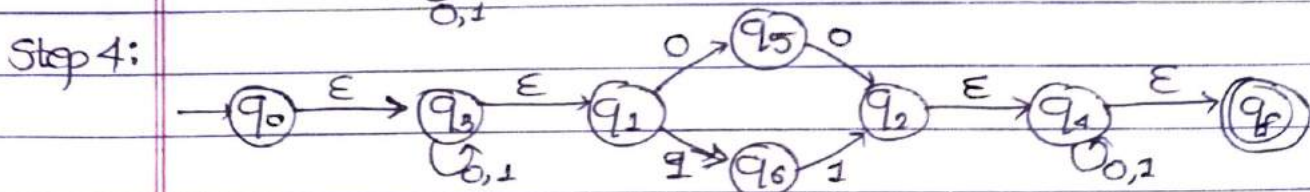
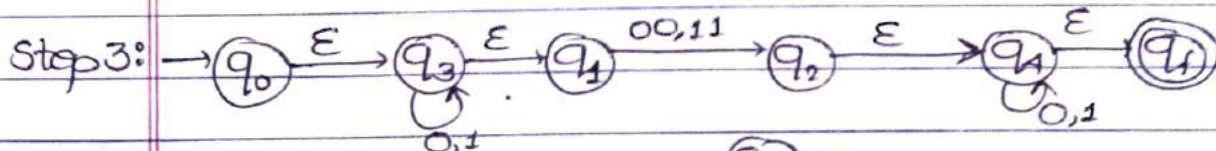
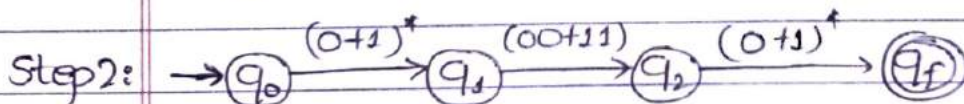
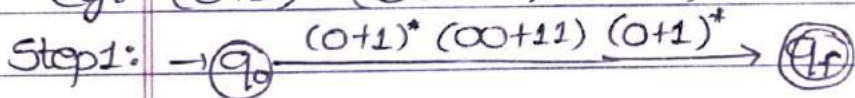


State diagram for DFA is:

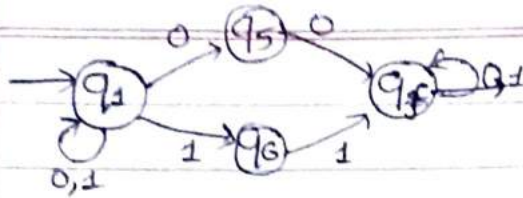
F.A from regular expression:

1. $a+b$ $\rightarrow q_1 \xrightarrow{a,b} q_f$
2. ab $\rightarrow q_1 \xrightarrow{a} q_2 \xrightarrow{b} q_f$
3. a^* $\rightarrow q_1 \xrightarrow{\epsilon} q_2 \xrightarrow{\epsilon} q_f$
 $q_2 \xrightarrow{a} q_2$
4. $(a+b)^*$ $\rightarrow q_1 \xrightarrow{\epsilon} q_2 \xrightarrow{\epsilon} q_f$
 $q_2 \xrightarrow{a,b} q_2$

eg: $(0+1)^* (00+11) (0+1)^*$

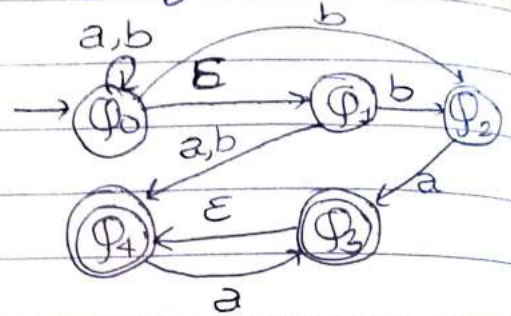


Step 5:



Q. Construct DFA for the following transition table:

State	Input	
	a	b
$\rightarrow q_0$	q_0	$\{q_0, q_1\}$
q_1	q_4	$\{q_2, q_4\}$
q_2	q_3	ϕ
q_3	ϕ	q_4
q_4	q_3	ϕ



$\Rightarrow$ Here, $\epsilon\text{-closure}(q_0) = \{q_0, q_1\}$

$\epsilon\text{-closure}(q_1) = \{q_1\}$

$\epsilon\text{-closure}(q_2) = \{q_2\}$

$\epsilon\text{-closure}(q_3) = \{q_3, q_4\}$

$\epsilon\text{-closure}(q_4) = \{q_4\}$

Now, δ' for DFA is defined as:

$$\begin{aligned} \text{(a)} \quad \delta'((q_0, q_1), a) &= \epsilon\text{-closure}(\delta(q_0, q_1), a) \\ &= \epsilon\text{-closure}(q_0 \cup q_4) \\ &= \{q_0, q_1, q_4\} \end{aligned}$$

$$\begin{aligned} \delta'((q_0, q_1), b) &= \epsilon\text{-closure}(\delta(q_0, q_1), b) \\ &= \epsilon\text{-closure}(q_0 \cup q_2 \cup q_4) \\ &= \{q_0, q_1, q_2, q_4\} \end{aligned}$$

$$\begin{aligned} \text{(b)} \quad \delta'((q_0, q_2, q_4), a) &= \epsilon\text{-closure}(\delta(q_0, q_2, q_4), a) \\ &= \epsilon\text{-closure}(q_0 \cup q_4 \cup q_3) \\ &= \{q_0, q_1, q_3, q_4\} \end{aligned}$$

$$\begin{aligned} \delta'((q_0, q_2, q_4), b) &= \epsilon\text{-closure}(\delta(q_0, q_2, q_4), b) \\ &= \epsilon\text{-closure}(q_0 \cup q_2 \cup q_4 \cup \phi) \\ &= \{q_0, q_1, q_2, q_4\} \end{aligned}$$

$$\begin{aligned} \textcircled{c} \quad \delta'((q_0 q_1 q_2 q_4), a) &= \epsilon\text{-closure}(\delta(q_0 q_1 q_2 q_4), a) \\ &= \epsilon\text{-closure}(q_0 \cup q_4 \cup q_3) \\ &= \{q_0 q_1 q_4 q_3\} \end{aligned}$$

$$\begin{aligned} \delta'((q_0 q_1 q_2 q_4), b) &= \epsilon\text{-closure}(\delta(q_0 q_1 q_2 q_4), b) \\ &= \epsilon\text{-closure}(q_0 \cup q_2 \cup q_4 \cup \phi) \\ &= \{q_0 q_1 q_2 q_4\} \end{aligned}$$

$$\begin{aligned} \textcircled{d} \quad \delta'(\delta(q_0 q_1 q_3 q_4), a) &= \epsilon\text{-closure}(\delta(q_0 q_1 q_3 q_4), a) \\ &= \epsilon\text{-closure}(q_0 \cup q_4 \cup q_3) \\ &= \{q_0 q_1 q_3 q_4\} \end{aligned}$$

$$\begin{aligned} \delta'(\delta(q_0 q_1 q_3 q_4), b) &= \epsilon\text{-closure}(\delta(q_0 q_1 q_3 q_4), b) \\ &= \epsilon\text{-closure}(q_0 \cup q_2 \cup q_4 \cup \phi) \\ &= \{q_0 q_1 q_2 q_4\} \end{aligned}$$

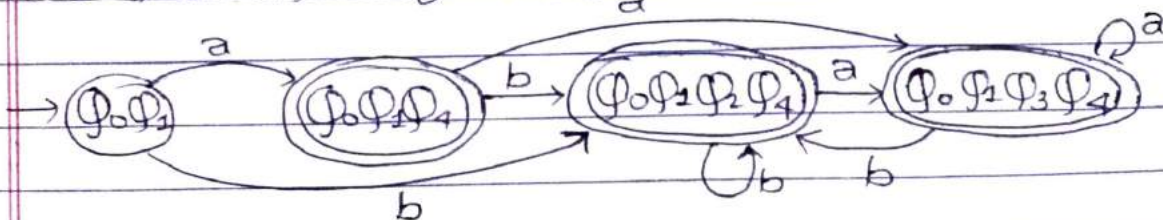
$$\begin{aligned} \textcircled{e} \quad \delta'(\delta(q_0 q_1 q_2 q_3 q_4), a) &= \epsilon\text{-closure}(\delta(q_0 q_1 q_2 q_3 q_4), a) \\ &= \epsilon\text{-closure}(q_0 \cup q_4 \cup q_3 \cup \phi) \\ &= \{q_0 q_1 q_3 q_4\} \end{aligned}$$

$$\begin{aligned} \delta'(\delta(q_0 q_1 q_2 q_3 q_4), b) &= \epsilon\text{-closure}(\delta(q_0 q_1 q_2 q_3 q_4), b) \\ &= \epsilon\text{-closure}(q_0 \cup q_1 \cup q_4 \cup \phi) \\ &= \{q_0 q_1 q_4\} \end{aligned}$$

$\therefore$ State table for DFA is as:

States	Input	
	a	b
$\rightarrow \{q_0 q_1\}$	$\{q_0 q_1 q_4\}$	$\{q_0 q_1 q_2 q_4\}$
$\{q_0 q_1 q_4\}$	$\{q_0 q_1 q_3 q_4\}$	$\{q_0 q_1 q_2 q_4\}$
$\{q_0 q_1 q_2 q_4\}$	$\{q_0 q_1 q_3 q_4\}$	$\{q_0 q_1 q_2 q_4\}$
$\{q_0 q_1 q_3 q_4\}$	$\{q_0 q_1 q_3 q_4\}$	$\{q_0 q_1 q_2 q_3 q_4\}$

Now, state diagram for DFA is as:



Q.1. Construct FA equivalent to $(ab+ac)^*b$:

⇒ Step 1: $\rightarrow q_0 \xrightarrow{(ab+ac)^*b} q_f$

Step 2: $\rightarrow q_0 \xrightarrow{\epsilon} q_1 \xrightarrow{\epsilon} q_2 \xrightarrow{b} q_f$ $q_1 \xrightarrow{a} q_2 \xrightarrow{b} q_2$
 $(ab+ac)^*$

Step 3: $\rightarrow q_0 \xrightarrow{\epsilon} q_1 \xrightarrow{\epsilon} q_2 \xrightarrow{b} q_f$
 $q_1 \xrightarrow{a} q_3 \xrightarrow{b} q_1$
 $q_1 \xrightarrow{a} q_4 \xrightarrow{c} q_1$

Step 4: $\rightarrow q_1 \xrightarrow{b} q_f$
 $q_1 \xrightarrow{a} q_3 \xrightarrow{b} q_1$
 $q_1 \xrightarrow{a} q_4 \xrightarrow{c} q_1$

2. Construct FA equivalent to $(alb)^* (abblatb)$.

⇒ Step 1: $\rightarrow q_0 \xrightarrow{(alb)^* (abblatb)} q_f$

Step 2: $\rightarrow q_0 \xrightarrow{(alb)^*} q_1 \xrightarrow{(abblatb)} q_f$

Step 3: $\rightarrow q_0 \xrightarrow{\epsilon} q_2 \xrightarrow{\epsilon} q_1 \xrightarrow{abb} q_f$
 $q_2 \xrightarrow{a,b} q_2$
 $q_1 \xrightarrow{atb} q_f$

Step 4: $\rightarrow q_0 \xrightarrow{\epsilon} q_2 \xrightarrow{\epsilon} q_1 \xrightarrow{a} q_3 \xrightarrow{b} q_4 \xrightarrow{b} q_f$
 $q_2 \xrightarrow{a,b} q_2$
 $q_1 \xrightarrow{a} q_5 \xrightarrow{b} q_4$
 $q_5 \xrightarrow{a} q_5$

Step 5: $\rightarrow q_1 \xrightarrow{a} q_3 \xrightarrow{b} q_4 \xrightarrow{b} q_f$
 $q_1 \xrightarrow{a,b} q_1$
 $q_1 \xrightarrow{a} q_5 \xrightarrow{b} q_4$
 $q_5 \xrightarrow{a} q_5$

Minimization of DFA:

eg:

Current state	Input	
	a	b
i. $\rightarrow q_0$	q_5	q_1
q_1	q_2	q_6
f. $*q_2$	q_2	q_0
q_4	q_5	q_7
q_5	q_6	q_2
q_6	q_4	q_6
q_7	q_2	q_6
q_3	q_6	q_2

$\Rightarrow$ Step 1: Remove states that can't be reached from initial states. Here, q_3 can't be reached. So, we remove q_3 from table. So, new table is:

C.S	Input	
	a	b
$\rightarrow q_0$	q_5	q_1
q_1	q_2	q_6
$*q_2$	q_2	q_0
q_4	q_5	q_7
q_5	q_6	q_2
q_6	q_4	q_6
q_7	q_2	q_6

Step 2: Divide the table into two sets:

a) Set containing final state:

$*q_2$ q_2 q_0

b) Set containing non-final state:

$\rightarrow q_0$ q_5 q_1
 q_1 q_2 q_6
 $*q_2, q_4$ q_5 q_7
 q_5 q_6 q_2
 q_6 q_4 q_6
 q_7 q_2 q_6

Step 3: (a) Considering set of non-final states:

(i) Here, R_2 and R_6 are same. So, we remove R_6 & replace q_7 with q_1 .

$\rightarrow q_0$	q_5	q_1
q_1	q_2	q_6
q_4	q_5	q_1
q_5	q_6	q_2
q_6	q_4	q_6

(ii) Here, R_1 and R_3 are same. So, we remove R_3 & replace q_4 with q_0 .

$\rightarrow q_0$	q_5	q_1
q_1	q_2	q_6
q_5	q_6	q_2
q_6	q_4	q_6

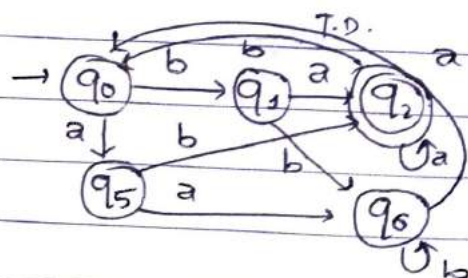
(b) Considering set of final states:

(i) Here, only one row is present. So, no removal of identical rows is necessary.

$*q_2$	q_2	q_0
--------	-------	-------

Combining set of final & non-final states:

$\rightarrow q_0$	q_5	q_1
q_1	q_2	q_6
q_5	q_6	q_2
q_6	q_0	q_6
$*q_2$	q_2	q_0



This is the req minimized DFA.

Properties of regular language or expression:

1. Commutativity : $L+R = R+L$
2. Associativity : $L+(R+S) = (L+R)+S$
 $L(RS) = (LR)S$
3. Identities : $\phi + R = R + \phi = R$, $\phi \cdot R = \phi$, $R \cdot \phi = \phi$, $R \cdot R = R$
4. Annihilator : $\phi R = R \phi = \phi$
5. Idempotent law : $L+L = L$
6. Distributive law : $L(R+S) = LR+LS$
7. Law of closure : $(R^*)^* = R^*$, $\phi^* = \phi$, $\epsilon^* = \epsilon$

Pumping lemma for regular set of language -

The pumping lemma gives a method of pumping many input string from a given string. In general, it gives a necessary condition for an input string to belong to a regular set.

Statement: Let $M = \{Q, \Sigma, \delta, q_0, F\}$ be a finite automata with n -states that accept the regular language L . Let w be the string such that $w \in L$ and $|w| \geq n$ then there exists x, y, z such that,

- i) $w = xyz$ ii) $y \neq \epsilon$ iii) $xy^iz \in L, i \geq 0$

Proof:

Let $w = a_1 a_2 a_3 \dots a_m, m \geq n$

$\delta(q_0, (a_1 a_2 a_3 \dots a_i)) = q_i$ = sequence of states in the path with path value $w = a_1 a_2 \dots a_m$.

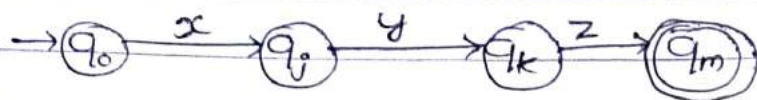
Now, the i/p string w can be decomposed into three sub-string such as

$$x = a_1 a_2 \dots a_j$$

$$y = a_{j+1} \dots a_k$$

$$z = a_{k+1} \dots a_m$$

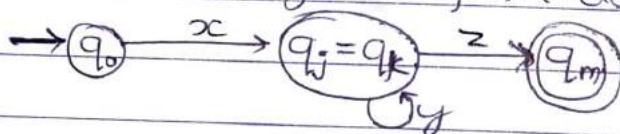
Since, we know that for any regular expression, there exists a path from initial to final state with path value in the given expression. For w , the FA can be constructed as:



Here, length of string, $|w|=3$

no. of states, $n=4$

On reading the string xyz ($|w|=m=3$), new states are added on the existing states i.e $n=4$. But by defⁿ of pumping lemma, we have, $|w| \geq n$. Thus, by Pigeon-Hole principle, there must be at least two states in Q . As there are only n distinct states defined but on applying the input string, the states become $n+1$. Thus, among various pair of repeated states, we take first as q_j and q_k for merging (i.e $q_j = q_k$) and hence the path with value w in the transition diagram of M as shown below:



Thus, $|w|=3, n=3$.

So, we have, $m \geq n$. Since, $w = xyz = a_1 a_2 \dots a_j a_{j+1} \dots a_k a_{k+1} \dots a_m$

So, we can write $0 \leq j < k \leq n$. This shows that $|xy| \leq n$.

Here, we have, $xy^i z \in L$ for each $i \geq 0$.

When $i=0$, $w = xz$ (accepted by FA)

When $i=1$, $w = xyz$ (" " ")

When $i=2$, $w = xy^2 z$ (" " ") and so on.

Here, we conclude that the condition is valid for all regular expression.

The figure shows the string accepted by FA M . Here, M start from initial state q_0 and on applying x it reaches state $q_i = q_k$ until application of y^i where $y \neq \epsilon$ & finally it reaches to final state q_m on applying z . Thus, the string of y can be formed several no. of times to generate new string. Hence $xy^iz \in L$.

As sequence of state in Q_1 is obtained by applying i/p symbol so $y \neq \epsilon$.

Application of pumping lemma - This theorem is used to prove that certain languages are not regular. The steps are :

1. Assume that L is regular. Let n be the no. of states in FA.
2. Choose a string w such that $|w| \geq n$. Use pumping lemma to write $w = xyz$ with $|xy| \leq n$ and $|y| > 0$ and $|y| \geq 1$.
3. Find a suitable integer i such that $xy^iz \notin L$. This contradicts our assumption. Hence, L is not regular.

Q1 Show that $L = \{0^n 1^n \mid n \geq 1\}$ is not regular.

Step 1: Assume L is regular.

Step 2: Let $w = 0^n 1^n$. By pumping lemma, we can write $w = xyz$

Let $n=7$, $w = 0000000 1111111$

Case a: y has only 0's i.e $w = \underbrace{0000000}_x \underbrace{0000000}_y \underbrace{1111111}_z$

Case b: y has only 1's i.e $w = \underbrace{0000000}_x \underbrace{1111111}_y \underbrace{1111111}_z$

Case c: y has both 0 & 1 i.e $w = \underbrace{0000000}_x \underbrace{0111111}_y \underbrace{1111111}_z$

Step 3: We want to find i so that $xy^i z \notin L$.

Suppose $i=2$.

a) In case a, $xy^2 z = 0000000000 1111111 = 0^9 1^7$

Since, no. of 1's and 0's are not equal i.e not in pattern. So, $xy^i z \notin L$ for $i=2$.

Also, in case a, $|xy| = 7 \leq 7$ (true)

b) In case b, $xy^2 z = 0000000 1111 1111111 = 0^7 1^9$

Since, no. of 1's and 0's are not equal i.e not in pattern. So, $xy^i z \notin L$ for $i=2$.

Also, in case b, $|xy| = 9 \leq 7$ (false)

c) In case c, $xy^2 z = 0000000101 1111111 = 0^8 1^8$

It's not in pattern. So, $xy^i z \notin L$.

Also, in case c, $|xy| = 8 \leq 7$ (false)

Thus, in all cases, we get $xy^i z \notin L$ for $i=2$. This contradicts our assumption. So, given language $L = \{0^n 1^n \mid n \geq 1\}$ is not regular.

2. Show that $L = \{ww \mid w \in \{ab\}^*\}$ is not regular.

→ Step 1: Assume L is regular.

Step 2: Suppose $w = a^n b$. So, $ww = a^n b a^n b$.

Then, by pumping lemma, we can write $ww = xyz$ such that $|xy| \leq n$ and $y \neq \Lambda$ and $|y| > 0$.

Let $n=7$, $ww = a^7 b a^7 b = \underbrace{aaaaaaa}_{x_1} \underbrace{ab}_{y_1} \underbrace{aaaaaaa}_{x_2} \underbrace{ab}_{y_2}$

Case a: y has only a i.e. $ww = \underbrace{aaaaaaa}_{x_1} \underbrace{aaaaaaa}_{y_1} \underbrace{ab}_{x_2} \underbrace{ab}_{y_2}$

Case b: y has only b i.e. $ww = \underbrace{aaaaaaa}_{x_1} \underbrace{a}_{y_1} \underbrace{ab}_{x_2} \underbrace{aaaaaaa}_{y_2} \underbrace{ab}_{z}$

Case c: y has both a and b i.e. $ww = \underbrace{aaaaaaa}_{x_1} \underbrace{abab}_{y_1} \underbrace{aaaaaaa}_{x_2} \underbrace{ab}_{y_2}$

We want to find i so that $xy^i z \notin L$.

Let $i=2$. Then,

a) In case a, $xy^2 z = \underbrace{aaaaa}_{x_1} \underbrace{aaaa}_{y_1} \underbrace{abaaaaaa}_{x_2} \underbrace{ab}_{y_2} = a^9 b a^7 b$

It's not in pattern, so $xy^i z \notin L$ for $i=2$.

Also, in case a, $|xy| = 7 \leq 7$ (true)

b) In case b, $xy^2 z = \underbrace{aaaaaa}_{x_1} \underbrace{abb}_{y_1} \underbrace{aaaaaa}_{x_2} \underbrace{ab}_{y_2} = a^7 b^2 a^7 b$

which is not in pattern. So, $xy^i z \notin L$ for $i=2$. $|xy| = 8 \leq 7$ (f)

c) In case c, $xy^2 z = \underbrace{aaaaaa}_{x_1} \underbrace{ababab}_{y_1} \underbrace{aaaaaa}_{x_2} \underbrace{ab}_{y_2} = a^7 b a b a b a^7 b$,

which is not in pattern. So, $xy^i z \notin L$ for $i=2$.

Also, $|xy| = 8 \leq 7$ (false)

Thus, for all cases, we get $xy^i z \notin L$ for $i=2$. This contradicts our assumption. Thus, given language is not regular.

3. Show that $L = \{a^p \mid p \text{ is prime}\}$ is not regular.

→ Step 1: Assume L is regular.

Step 2: Suppose $w = a^p$. Then, by pumping lemma, $w = a^p$ such that $|xy| \leq n$ and $|y| > 0$.

Let $p=5$. Then, $w = \underbrace{aaaaa}_{x_1} \underbrace{a}_{y_1}$

The only possible case is that y has only a . So,
 $w = \underbrace{aaaaa}_x \underbrace{a}_y \underbrace{a}_z$.

We want to find i for $xy^iz \notin L$.

Suppose $i=2$. Then, $w = aaaaaa = a^6$

where 6 is not a prime number. Also, $|xy| = 3 \leq 7$ (true)

Thus, we get $xy^iz \notin L$ for $i \in \mathbb{Z}$. This contradicts our assumption. Thus, $L = \{a^p \mid p \text{ is prime}\}$ is not regular.

Closure properties of regular language -

1. The union and intersection of two regular language is regular.
2. The difference between two regular language is regular.
3. Complement of regular language is regular.
4. The concatenation of regular language is regular.
5. The closure operation (iteration) on RL is regular.

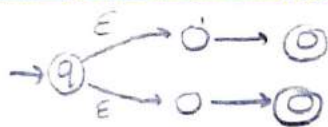
Theorem 1: If L_1 and L_2 are two regular languages, then $L_1 \cup L_2$ is also regular.

Let $M_1 = (Q_1, \Sigma_1, \delta_1, q_1, F_1)$ be NFA that accepts language L_1 i.e $L(M_1)$ and $M_2 = (Q_2, \Sigma_2, \delta_2, q_2, F_2)$ be NFA that accepts language L_2 i.e $L(M_2)$.

Assume Q_1 and Q_2 are disjoint sets.

Now, we construct NFA, $M = (Q, \Sigma, \delta, q, F)$ where such that it accepts $L = L(M_1) \cup L(M_2)$ where q is the new state not in Q_1 or Q_2 where

$Q = Q_1 \cup Q_2 \cup \{q\}$, $\Sigma = \Sigma_1 \cup \Sigma_2 \cup \{\epsilon\}$, $\delta = \delta_1 \cup \delta_2 \cup [(q, \epsilon) \rightarrow q_1, (q, \epsilon) \rightarrow q_2]$, $F = F_1 \cup F_2$.



DATE : ___/___/___

PAGE : _____

M begins any computation non-deterministically choosing either the initial state of M_1 or M_2 and interpret M initiates either M_1 or M_2 .

If $\omega \in \Sigma^*$, then $(q, \omega) \xrightarrow[M]{\text{many states are changing}} (p, \epsilon)$ for $p \in F$ if & only if $(q, \omega) \xrightarrow[M_1]{*} (p, \epsilon)$ for $p \in F_1$ or $(q, \omega) \xrightarrow[M_2]{*} (p, \epsilon)$ for $p \in F_2$.

Hence, $L_1 \cup L_2$ i.e. $L(M) = L(M_1) \cup L(M_2)$ is regular.

Theorem 2: If L & M are ~~also~~ regular, then LM is also regular.

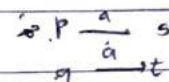
$\Rightarrow$ Let L be recognized by DFA $A_L = \{Q_L, \Sigma_L, \delta_L, q_L, F_L\}$ & M by DFA $A_M = \{Q_M, \Sigma_M, \delta_M, q_M, F_M\}$. We assume that alphabets of both automata are same i.e. Σ is the union of Σ_L and Σ_M if they are different.

We construct an automata A such that $A = (Q, \Sigma, \delta, q, F)$ where, $Q = Q_L \times Q_M$, $\Sigma = \Sigma_L \cup \Sigma_M$, $\delta = (\delta_L, \delta_M)$, $q = (q_L, q_M)$, $F = F_L \times F_M$.

If A_L goes from state 'p' to 'q' on reading a and A_M goes from state 'q' to 't' on reading a, then A will go from state (p, q) to (s, t) on reading a. i.e. $\delta((p, q), a) = (\delta_L(p, a), \delta_M(q, a))$

A accepts string ' ω ' if and only if $\delta_{LM}((q_L, q_M), \omega)$ is a pair of accepting states i.e. $\delta_L(q_L, \omega)$ is in F_L and $\delta_M(q_M, \omega)$ is in F_M .

Thus, ω is accepted by A if and only if ω is accepted by A_L and A_M both.



$$\rightarrow M_1 \stackrel{\circ}{\circ} \xrightarrow{E} M_2 \circ$$

DATE: ___/___/___
PAGE: ___

Theorem 3: If L_1 and L_2 are regular, then $L_1 \cdot L_2$ is also regular.

Let $M_1 = (Q_1, \Sigma_1, \delta_1, q_1, F_1)$ be the NFA that accepts regular language L_1 ^{i.e. $L_1 = L(M_1)$} and $M_2 = (Q_2, \Sigma_2, \delta_2, q_2, F_2)$ be NFA that accepts L_2 i.e. $L_2 = L(M_2)$.

Now, we construct NFA, $M = (Q, \Sigma, \delta, q, F)$ such that it can accept the language $L_1 \cdot L_2$ i.e. $L = L(M) = L(M_1) \cdot L(M_2)$ where $Q = Q_1 \cup Q_2$, $\Sigma = \Sigma_1 \cup \Sigma_2 \cup \{\Lambda\}$, $\delta = \delta_1 \cup \delta_2 \cup \{(F_1, \Lambda) \rightarrow q_2\}$, $q = q_1$, $F = F_2$.

Formally, $(q, \omega) \xrightarrow{*}_M (p, \Lambda)$ for some $p \in F$ if and only if there are $\omega_1, \omega_2 \in \Sigma^*$ and there is $p_1 \in F_1$ such that $\omega = \omega_1 \cdot \omega_2$ and $(q_2, \omega_2) \xrightarrow{*}_{M_2} (p_2, \Lambda)$ for $p_2 \in F_2$ and $(q_1, \omega_1) \xrightarrow{*}_{M_1} (p_1, \Lambda)$ for some $p_1 \in F_1$. Hence, $L_1 \cdot L_2$ is also regular.

Theorem 4: The closure (iteration) i.e. Kleene closure operation on regular language is also regular.

Let $M_1 = (Q_1, \Sigma_1, \delta_1, q_1, F_1)$ be NFA that accepts L i.e. $L = L(M_1)$.

Now, we construct NFA, $M = (Q, \Sigma, \delta, q, F)$ such that it can accept closure of language i.e. $L = L(M) = L(M_1)^*$ where,

$Q = Q_1 \cup \{q\}$, $\Sigma = \Sigma_1 \cup \{\Lambda\}$, $\delta = \delta_1 \cup [(q, \Lambda) \rightarrow q_1, (q_1, \Lambda) \rightarrow F_1, (F_1, \Lambda) \rightarrow q]$, $q = \text{start state}$, $F = F_1 \cup \{q\}$

Here, M consists all states of M_1 & all the transition of M_1 and also any final state of M_1 is also the final state of M . The new initial state q is also final state so that ϵ or

Λ is also accepted.

It follows the inspection of M that if $\omega \in L(M)$ then either $\omega = \{\Lambda\}$ or $\omega = \{\omega_1, \omega_2, \dots, \omega_k\}$ where for $k \geq 1$, where $i = 1, 2, \dots, k$, there is $F_i \in F$ such that $(q_1, \omega) \xrightarrow[M_1]{*} (F_1, \epsilon)$. Hence, $\omega \in L(M_1)^*$, So, $L(M) = L(M_1)^*$

Theorem 5: If L is regular language over Σ , then $\bar{L} = \Sigma^* \setminus L$ is also regular.

Let L be accepted by DFA, $A = (Q, \Sigma, \delta, q_0, F)$. Then, $\bar{L} = L(B)$ where B is DFA that accepts $\bar{L}$ where $B = (Q, \Sigma, \delta, q_0, Q \setminus F)$. Here, B is exactly like A but the accepting states i.e. final states of A have become non-accepting state of B and vice-versa. Then, ω is in $L(B)$ if and only if $\delta(q_0, \omega)$ is in $(Q \setminus F)$ state which occurs only if ω is not in $L(A)$.

Theorem 6: If L and M are regular language, then the difference of L and M is also a regular language.

Decision algorithm for regular set or RL:

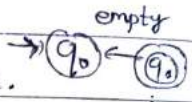
1. The membership problem - Membership is a property to verify an arbitrary string is accepted by a finite automaton or not.

Let w be a string defined over alphabet and M be a finite automata that accepts string over the alphabet. If there ~~is~~ exists a transition path in M , which starts at initial state & ends in any final state, the ' w ' is the member of M .

2. The emptiness and non-emptiness -

Step 1: Select the states that cannot be reached from initial state & remove them.

Step 2: If the resulting machine contains atleast one final state, the FA accepts non-empty language.

Step 3: Otherwise, FA accepts empty language. 

3. The finiteness and infiniteness -

Step 1: Select the states that cannot be reached from initial state & remove them.

Step 2: Select the states from which we cannot reach final state & remove them (dead states)

Step 3: If the resulting machine contains loop or cycle then FA accepts infinite language.

Step 4: If the resulting machine does not contain loop or cycle, then FA accepts finite language.

CHAPTER 3

CONTEXT FREE LANGUAGE

DATE : _/ _/ _

PAGE : _

Context free language is the language generated by context free grammar. It is accepted by pushdown automata.

Context free grammar - A CFG is a quadruple defined as $G = (V, \Sigma, P, S)$ where

V = set of variable / non-terminal symbols.

Σ = set of terminal symbols

P = production rule, S = starting state

* Production rule is in the form of $A \rightarrow \alpha$.
where, $\alpha \in (V \cup \Sigma)^*$

It is called context free because it is free from any context.

eg- Consider $S \xrightarrow{a} aaAb | \epsilon$ Here, S, A are variables and a, b are terminal symbols.

$S \rightarrow aaabAb$ ($A \rightarrow abAb$)

$S \rightarrow aaab\epsilon b$ ($A \rightarrow \epsilon$)

$S \rightarrow aaabbb$.

If derivation obtained by from any grammar is $S \xrightarrow{A \rightarrow abAb | \epsilon} aaAb | \epsilon$, then replacement of A by any terminal is independent on previous or next symbol to be read.

Q.1. Construct CFG for language having any no of a 's over set $\{a\}$.

$\Rightarrow$ The r.e is a^* .

$\therefore$ The production rule is $S \rightarrow aS | \epsilon$

So, CFG for r.e $= a^*$ is $G = (V, \Sigma, P, S)$ where

$V = \{S\}$, $\Sigma = \{\epsilon, a\}$, $P = \{S \rightarrow aS | \epsilon\}$, $S = S$.

eg- If we want to derive 'aaa', we can use production rule as $S \rightarrow aS | \epsilon$ ($S \rightarrow aS$)

$\rightarrow aaaS$ ($S \rightarrow aS$)

$\rightarrow aaas$ ($S \rightarrow \epsilon$)

$\rightarrow aaa$

2. Construct CFG for r.e $(0+1)^*$.

$\Rightarrow$ The r.e is $r.e = (0+1)^*$

The production rule is $S \rightarrow 0S | 1S | \epsilon$

So, CFG for given r.e is $G = (V, \Sigma, P, S)$ where

$V = \{S\}$, $\Sigma = \{0, 1, \epsilon\}$, $P = \{S \rightarrow 0S | 1S | \epsilon\}$, $S = S$

eg- If we want to derive '010', then we can use production rule as $S \rightarrow 0S | 1S | \epsilon$ ($S \rightarrow 1S$)

$\rightarrow 01S$ ($S \rightarrow 0S$)

$\rightarrow 010S$ ($S \rightarrow \epsilon$)

$\rightarrow 010$

3. Construct a CFG for $L = w c w^T$, $w \in \{a, b\}^*$.

$\Rightarrow$ The production rule for given r.e is $S \rightarrow c, S \rightarrow aSa | bSb$

So, the CFG is $G = \{V, \Sigma, P, S\}$, where

$V = \{c\}$, $\Sigma = \{a, b\}$, $P = \{S \rightarrow aSa | bSb\}$, $S = S$.

eg- If we want to derive 'abcba', then we can use production rule as, $S \rightarrow aSa | bSb$ ($S \rightarrow bSb$)

$\rightarrow abSba$ ($S \rightarrow c$)

$\rightarrow abcba$

4. for L containing atleast two a's defined over $\Sigma = \{a, b\}$.

$\Rightarrow$ The r.e is $r.e = (a+b)^* a (a+b)^* a (a+b)^*$

The production rule is $A \rightarrow AaA | A | bA | \epsilon$

$S \rightarrow AaAaA$

$S \rightarrow aSb| \epsilon$

5. For $L = \{a^n b^n \mid n \geq 1\}$

⇒ The production rule is $S \rightarrow AB, A \rightarrow aA|a, B \rightarrow bB|b$.

∴ The CG is $G = \{V, \Sigma, P, S\}$ where $V = \{S, A, B\}$,
 $\Sigma = \{a, b\}$, $P = \{S \rightarrow AB, A \rightarrow aA|a, B \rightarrow bB|b\}$, $S = S$.

6. $L = \{a^m b^n \mid n \geq 1, m \geq 0\}$

⇒ The production rule is $S \rightarrow AB, A \rightarrow aA|a| \epsilon, B \rightarrow bB|b$.

∴ CG is $G = \{V, \Sigma, P, S\}$ where $V = \{S, A, B\}$, $\Sigma = \{a, b, \epsilon\}$,
 $P = \{S \rightarrow AB, A \rightarrow aA|a| \epsilon, B \rightarrow bB|b\}$, $S = S$.

7. $x.e = (0+1)^* 000 (0+1)^*$ $S \rightarrow A000A$ $A \rightarrow 0A|1A| \epsilon$

⇒ The production rule is $S \rightarrow A000A, A \rightarrow 0A|1A| \epsilon, B \rightarrow 0B|1B| \epsilon$

∴ CG is $G = \{V, \Sigma, P, S\}$, where $V = \{S, A, B\}$, $\Sigma = \{0, 1, \epsilon\}$,
 $P = \{S \rightarrow A000A, A \rightarrow 0A|1A| \epsilon, B \rightarrow 0B|1B| \epsilon\}$, $S = S$.

8. for a string containing the first and last symbol different for $\Sigma = \{0, 1\}$.

⇒ The production rule is $S \rightarrow 0A1|1A0| \epsilon, A \rightarrow 0A|1A| \epsilon$.

∴ CG is $G = \{V, \Sigma, P, S\}$ where $V = \{S, A\}$, $\Sigma = \{0, 1, \epsilon\}$,
 $P = \{S \rightarrow 0A1|1A0, A \rightarrow 0A|1A| \epsilon\}$, $S = S$.

Derivation of CFG: Process of deriving strings from the given CFG using rules.

eg- Derive a^4 from grammar.

⇒ Terminal: a , Non-terminal: S

Production rule: $S \rightarrow aS| \epsilon$ ($S \rightarrow aS$)

→ aas ($aas \rightarrow aS$)

→ $aaas$ ($S \rightarrow aS$)

→ $aaaaS$ ($S \rightarrow \epsilon$)

→ $aaaa$.

Since a^1 i.e. $aaaa$ is derived, ~~so~~ a^1 is accepted by grammar.

eg: Let $G = \{ (S, C), (a, b), P, S \}$ where P is $S \rightarrow aCa, aCa/b$. Find $L(G)$

$\Rightarrow$ Starting from $S \rightarrow aCa$ ($C \rightarrow b$)

$$\therefore S \rightarrow aba$$

$$\therefore aba \in L(G).$$

$$\text{And, } S \rightarrow aCa \rightarrow aaCa \rightarrow aabaa$$

$$\therefore aabaa \in L(G).$$

Similarly, $aaabaaa \in L(G)$, $aaaabaaaa \in L(G)$.

This goes infinite and we derive $a^n b a^n \in L(G)$.

$L = \{ a^n b a^n, n \geq 1 \}$. This is the required CFL.

* Find $L(G)$. $S \rightarrow aS, S \rightarrow bS, S \rightarrow a, S \rightarrow b$.

$\Rightarrow$ Here, $S \rightarrow aS$ ($a \in L(G)$)

$$S \rightarrow b \quad (b \in L(G))$$

$$S \rightarrow aS \rightarrow aa. \text{ Similarly, } aaa \in L(G).$$

$$S \rightarrow bS \rightarrow ba. \text{ Similarly, } baaa \in L(G)$$

$$S \rightarrow aS \rightarrow ab$$

$$S \rightarrow bS \rightarrow bb$$

$\therefore$ Required language, $L = (a+b)^+$

* Find the language and derive 'abbaaba' from following grammar. $S \rightarrow XaaX, X \rightarrow aX | bX | \epsilon$.

$\Rightarrow$ Starting from S , $S \rightarrow XaaX$ ($X \rightarrow aX$)

$$\rightarrow aXaaX \quad (X \rightarrow bX)$$

$$\rightarrow abXaaX \quad (X \rightarrow bX)$$

$$\rightarrow abbXaaX \quad (X \rightarrow \epsilon)$$

$$\rightarrow abbaaX \quad (X \rightarrow bX)$$

$$\rightarrow abbaabX \quad (X \rightarrow aX)$$

$$\rightarrow abbaaba \quad (X \rightarrow \epsilon)$$

Reqⁿ language, $L = \{(a+b)^* aa (a+b)^*\}$

Q. If G is grammar, $G = \{V, \Sigma, P, S\}$, $V = \{A, B, S\}$, $\Sigma = \{a, b\}$, $S = S$, P is $S \rightarrow AaB$, $A \rightarrow baA|a$, $B \rightarrow abB|b$. Check whether the given string is accepted by grammar or not. (i) baaab (ii) aab (iii) baaabb

solⁿ Given, $P = \{S \rightarrow AaB, A \rightarrow baA|a, B \rightarrow abB|b\}$

i) baaab

$\Rightarrow S \rightarrow \cancel{ba}AaB \quad (A \rightarrow baA)$
 $\rightarrow baAaB \quad (A \rightarrow a)$
 $\rightarrow baaaB \quad (B \rightarrow b)$
 $\rightarrow baaab$

Since the string can be derived, it is accepted by grammar G .

ii) aab

$\Rightarrow S \rightarrow AaB \quad (A \rightarrow a)$
 $\rightarrow aaB \quad (B \rightarrow b)$
 $\rightarrow aab$

Since, the string can be derived, it is accepted.

iii) baaabb

$\Rightarrow S \rightarrow AaB \quad (A \rightarrow baA)$
 $\rightarrow baAaB \quad (A \rightarrow a)$
 $\rightarrow baaaB \quad (B \rightarrow b)$
 $\rightarrow baaab$

Since, the string cannot be derived, it's not accepted.

Derivation of CFG: Can be classified as:

- 1) Leftmost: if we apply production rule to leftmost variable at any time.
- 2) Rightmost: if we apply production rule to rightmost variable at any time.
- 3) Mixed - combination of leftmost and rightmost derivation.

Derivation tree/ Parse tree - Derivation of strings

A derivation tree for CFG, $G = \{V, \Sigma, P, S\}$ satisfies following conditions:

- 1) Every vertex has a label which is a variable or terminal or Λ .
- 2) The root has label S .
- 3) The label of an internal vertex is always a var.
- 4) If the vertices $n_1, n_2, \dots, n_k$ written with labels $X_1, X_2, \dots, X_k$ which are the sons of vertex n with label A , then $A \rightarrow X_1 X_2 \dots X_k$ is a production rule.

eg- $S \rightarrow bSb/a/b$

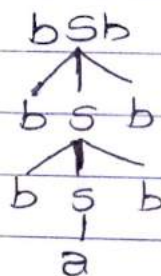
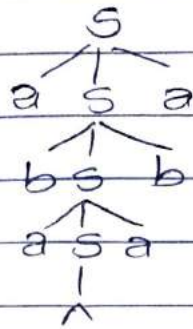


fig- Derivation tree

abaaba

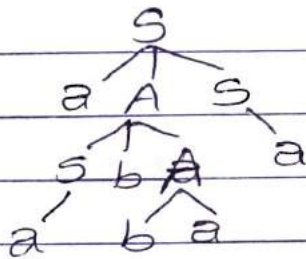
Q.1. Draw a derivation tree for the string a^3b^3 for the given grammar G , where $P = \{S \rightarrow aSa, S \rightarrow bSb, S \rightarrow a|b \in \Sigma\}$.

- ⇒ Starting from S, $S \rightarrow asa$ ($S \rightarrow bsb$)
 $\rightarrow abSba$ ($S \rightarrow asa$)
 $\rightarrow abasaba$ ($S \rightarrow \epsilon$)
 $\rightarrow abaaba$



2. Consider a grammar G which has production rule, $S \rightarrow aAS|a$, $A \rightarrow SbA|ss|ba$. Construct a parse tree that yields $aabbaa$.

- ⇒ Starting from S, $S \rightarrow aAS$ ($A \rightarrow SbA$)
 $\rightarrow aSbAS$ ($S \rightarrow a$)
 $\rightarrow aabAS$ ($A \rightarrow ba$)
 $\rightarrow aabbas$ ($S \rightarrow \epsilon$)
 $\rightarrow aabbaa$.



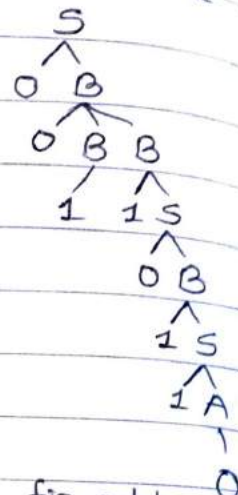
3. Let ' G ' be the grammar, $S \rightarrow 0B|1A$, $A \rightarrow 0|0S|1AA$, $B \rightarrow 1|1S|0BB$. Find the string 00110101 by
- Leftmost derivation
 - Rightmost derivation
 - Mixed derivation
 - Make parse tree for each derivation.

i) Starting from S, $S \rightarrow 0B$ ($B \rightarrow 0BB$)
 $\rightarrow 00BB$ ($B \rightarrow 1S$)
 $\rightarrow 001SB$ ($S \rightarrow 1A$)
 $\rightarrow 0011AB$ ($A \rightarrow 0S$)
 $\rightarrow 00110SB$ ($S \rightarrow 1A$)
 $\rightarrow 001101AB$ ($A \rightarrow 0$)
 $\rightarrow 0011010B$ ($B \rightarrow 1$)
 $\rightarrow 00110101$



fig-leftmost parse tree

ii) Starting from S, $S \rightarrow 0B$ ($B \rightarrow 0BB$)
 $\rightarrow 00BB$ ($B \rightarrow 1S$)
 $\rightarrow 00B1S$ ($S \rightarrow 0B$)
 $\rightarrow 00B10B$ ($B \rightarrow 1$)
 $\rightarrow 00B101$ ($B \rightarrow 1S$)
 $\rightarrow 001S101$ ($S \rightarrow 1A$)
 $\rightarrow 0011A101$ ($A \rightarrow 0$)
 $\rightarrow 00110101$



iii) $S \rightarrow 0B$ ($B \rightarrow 0BB$)
 $\rightarrow 00BB$ ($B \rightarrow 1, B \rightarrow 1S$)
 $\rightarrow 0011S$ ($S \rightarrow 0B$)
 $\rightarrow 00110B$ ($B \rightarrow 1S$)
 $\rightarrow 001101S$ ($S \rightarrow 0B$)
 $\rightarrow 0011010B$ ($B \rightarrow 1$)
 $\rightarrow 00110101$

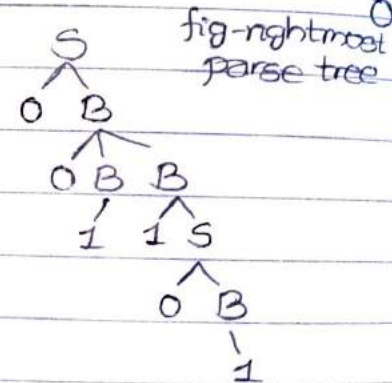


fig-rightmost parse tree

fig-mixed parse tree

4. Let G be the grammar, $S \rightarrow aB|bA$, $A \rightarrow aS|bAA|a$, $B \rightarrow bS|aBB|b$. Find the string $aaabbabbba$ by

- i) leftmost derivation
- ii) rightmost derivation
- iii) mixed derivation
- iv) make a parse tree for each derivation.

⇒ i) Starting from S, $S \rightarrow aB$ ($B \rightarrow aBB$)

$\rightarrow aaBB$ ($B \rightarrow aBB$)

$\rightarrow aaaSBBB$ ($B \rightarrow b$)

$\rightarrow aaabBB$ ($B \rightarrow b$)

$\rightarrow aaabbB$ ($B \rightarrow aBB$)

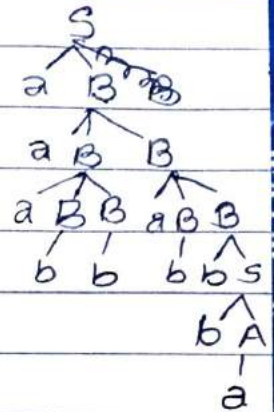
$\rightarrow aaabbabbB$ ($B \rightarrow bA$)

$\rightarrow aaabbabbB$ ($B \rightarrow bs$)

$\rightarrow aaabbabbbs$ ($s \rightarrow bA$)

$\rightarrow aaabbabbbsA$ ($A \rightarrow a$)

$\rightarrow aaabbabbbsa$



ii) $S \rightarrow aB$ ($B \rightarrow aBB$)

$\rightarrow aaBB$ ($B \rightarrow aBB$)

$\rightarrow aaBaBB$ ($B \rightarrow bs$)

$\rightarrow aaBaBbs$ ($s \rightarrow bA$)

$\rightarrow aaBaBbsbA$ ($A \rightarrow a$)

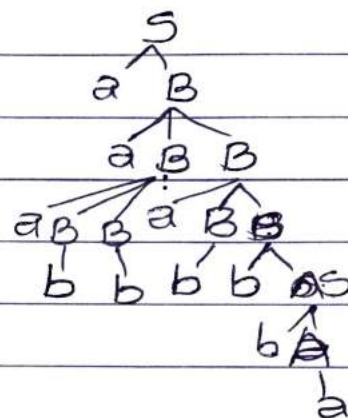
$\rightarrow aaBaBbsba$ ($B \rightarrow b$)

$\rightarrow aaBaabbsba$ ($B \rightarrow aBB$)

$\rightarrow aaBaabbsba$ ($B \rightarrow b$)

$\rightarrow aaBaabbsba$ ($B \rightarrow b$)

$\rightarrow aaBaabbsba$



iii) $S \rightarrow aB$ ($B \rightarrow aBB$)

$\rightarrow aaBB$ ($B \rightarrow aBB, B \rightarrow bs$)

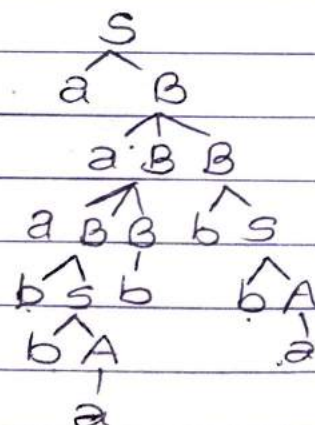
$\rightarrow aaaSBBbs$ ($s \rightarrow bA, B \rightarrow b$)

$\rightarrow aaaSbbbsA$ ($B \rightarrow b(s, A \rightarrow a)$)

$\rightarrow aaaSbbbsA$ ($s \rightarrow bA$)

$\rightarrow aaaSbbbsA$ ($A \rightarrow a$)

$\rightarrow aaaSbbbsa$



Q. Consider CFG with following pr:-

$S \rightarrow ASA|B$, $B \rightarrow acb|bca$, $C \rightarrow ACA|A$, $A \rightarrow alb$

- i) What are the variables and terminals in G?
- ii) Give the string of length '7' in $L(G)$.
- iii) Are the following string is in language of G?

1. aaa 2. bbb 3. aba 4. abb

- iv) True or false, $C \rightarrow bab$
- v) True or false, $C \xrightarrow{*} bab$
- vi) True or false, $C \xrightarrow{*} AAA$

⇒ i) The variables are $V = \{S, A, B, C\}$ and terminals are $\Sigma = \{a, b\}$

ii) Starting from $S \rightarrow ASAB$ ($A \rightarrow B \rightarrow \overset{acb}{ASA}$)
 $\rightarrow AASAA \rightarrow \overset{acb}{ACb}$ ($C \rightarrow ACA$)
 $\rightarrow aACAb$ ($C \rightarrow ACA$)
 $\rightarrow aAACAAb$ ($A \rightarrow a, C \rightarrow A$)
 $\rightarrow aaaaAaab$ ($A \rightarrow b$)
 $\rightarrow aaaaabaab$

iii) 1. $S \rightarrow ASA$ ($A \rightarrow a$)
 $\rightarrow aSa$ ($S \rightarrow B$)
 $\therefore \overset{a}{aaa}$ can't be derived, it's not in $L(G)$.

2. bbb $S \rightarrow B$ (bca)
 $\rightarrow bca \quad \therefore bbb \notin L(G)$.

3. aba : $S \rightarrow \overset{a}{ASA}$ ($A \rightarrow a$)
 $\rightarrow aSa \quad \therefore aba \notin L(G)$.

4. abb : $S \rightarrow acb$ ($C \rightarrow A$)
 $\rightarrow aAb$ ($A \rightarrow b$)
 $\rightarrow abb \in L(G)$

iv) $C \rightarrow bab$ (false)

v) $C \rightarrow ACA$ ($A \rightarrow b, C \rightarrow A$) $\rightarrow bAb$ ($A \rightarrow a$) $\rightarrow bab$
 $\therefore C \xrightarrow{*} bab$ (true)

vi) $C \rightarrow ACA$ ($C \rightarrow A$) $\rightarrow AAA$
 $\therefore C \xrightarrow{*} AAA$ (true)

Ambiguity in CG:

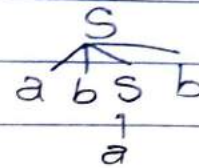
A terminal string $w \in L(G)$ is ambiguous if there exists two or more than two leftmost derivation or rightmost derivation or if there exists two or more than two derivation tree or parse tree.

eg- Show that the grammar

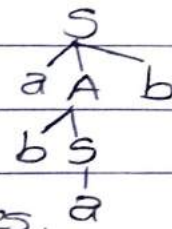
$$S \rightarrow a | abSb | aAb$$

$A \rightarrow bS | aAAb$ is ambiguous.

1st method: $S \rightarrow abSb$ ($S \rightarrow a$)
 $\rightarrow abab$



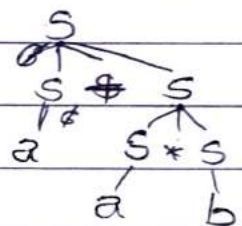
2nd method: $S \rightarrow aAb$ ($A \rightarrow bS$)
 $\rightarrow abSb$ ($S \rightarrow a$)
 $\rightarrow abab$



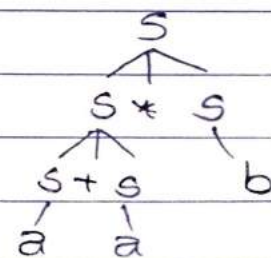
Since there exists two parse trees, the given grammar G is ambiguous.

Show that the grammar $G = \{S, (a, b, +, *), P, S\}$ where P is $S \rightarrow S+S | S*S | a | b$. Show the string $a+a*b$ is ambiguous.

We have, $P: S \rightarrow S+S$ ($S \rightarrow a$)
 $\rightarrow a+S$ ($S \rightarrow S*S$)
 $\rightarrow a+S*S$ ($S \rightarrow a$)
 $\rightarrow a+a*S$ ($S \rightarrow b$)
 $\rightarrow a+a*b$



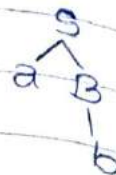
2nd method: $S \rightarrow S*S$ ($S \rightarrow S+S$)
 $\rightarrow S+S*S$ ($S \rightarrow a$)
 $\rightarrow a+S*S$ ($S \rightarrow a$)
 $\rightarrow a+a*S$ ($S \rightarrow b$)
 $\rightarrow a+a*b$



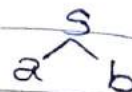
Since there exists two parse trees, the given string is ambiguous.

Show that grammar $S \rightarrow aB|ab, B \rightarrow ABb|b, A \rightarrow aAB|a$ is ambiguous.

$\Rightarrow$ 1st method: $S \rightarrow aB$ ($B \rightarrow ABb$)
 $\rightarrow aABb$ ($A \rightarrow a$) $\rightarrow ab$
 $\rightarrow aABb$ ($B \rightarrow b$)
 $\rightarrow aabb$



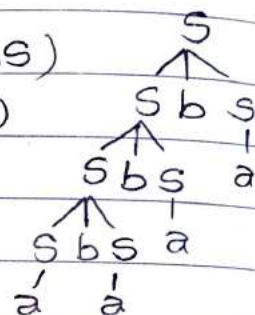
2nd method: $S \rightarrow ab$



Since there exists two parse trees, the grammar is ambiguous.

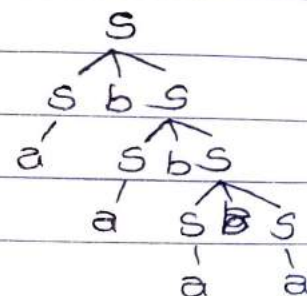
If G is grammar $S \rightarrow Sbs|a$. Show that G is ambiguous for string "abababa".

$\Rightarrow$ 1st method: $S \rightarrow aSbs$ ($S \rightarrow a$)
 $\rightarrow Sbsbs$ ($S \rightarrow Sbs$)
 $\rightarrow Sbsbsbs$ ($S \rightarrow a$)
 $\rightarrow abababa$



2nd method: $S \rightarrow Sbs$ ($S \rightarrow Sbs$)

$\rightarrow Sbsbs$ ($S \rightarrow Sbs$)
 $\rightarrow Sbsbsbs$ ($S \rightarrow a$)
 $\rightarrow Sbsbsba$ ($S \rightarrow a$)
 $\rightarrow Sbsbaba$ ($S \rightarrow a$)
 $\rightarrow Sbababa$ ($S \rightarrow a$)
 $\rightarrow abababa$



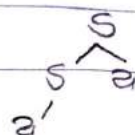
Since there exists two different parse trees, the string is ambiguous.

Consider $S \rightarrow aS(Sa|a)$ is ambiguous.

$\Rightarrow$ 1st method: $S \rightarrow aS$ ($S \rightarrow a$)
 $\rightarrow aa$



2nd method: $S \rightarrow Sa$ ($S \rightarrow a$)
 $\rightarrow aa$

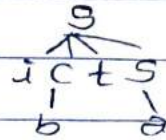


Since there exists two parse trees, the string aa accepted by G is ambiguous.

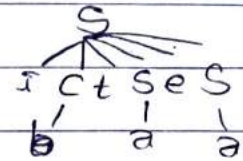
Check whether the grammar is ambiguous or not.

→ $S \rightarrow iCtS / iCtSeS / a, C \rightarrow b.$

⇒ 1st method: $S \rightarrow iCtS$ ($C \rightarrow b$)
 $\rightarrow ibtS$ ($S \rightarrow a$)
 $\rightarrow ibta$



2nd method: $S \rightarrow iCtSeS$ ($C \rightarrow b$)
 $\rightarrow ibtSeS$ ($S \rightarrow a$)
 $\rightarrow ibtaes$

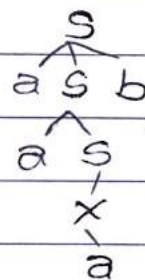
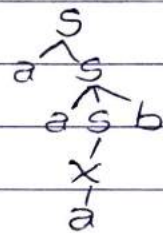


Since the same string can't be derived from two methods, the grammar is not ambiguous.

$S \rightarrow aS / aSb / X, X \rightarrow Xa / a$ is ambiguous.

→ 1st method: $S \rightarrow aS / aSb / X$ ($X \rightarrow S \rightarrow aS$) ($S \rightarrow X$) ($S \rightarrow aSb$)
 $\rightarrow aXb$ ($X \rightarrow a$) ($S \rightarrow (S \rightarrow X)$)
 $\rightarrow aaxb$ ($X \rightarrow a$)
 $\rightarrow aaab$

2nd method: $S \rightarrow aSb$ ($S \rightarrow aS$)
 $\rightarrow aaSb$ ($S \rightarrow X$)
 $\rightarrow aaab$



Since two different parse tree are derived, the string 'aaab' is ambiguous.

G for 'PALINDROMES' is $S \rightarrow aSa / bSb / a / b / \epsilon$. Check if G is ambiguous.

→ 1st method : $S \rightarrow aSa$ ($S \rightarrow b$)
→ aba

2nd method : $S \rightarrow bSb$ ($S \rightarrow a$)
→ bab

S
↑
a S a
|
b

S
↑
b S b
|
a

Since two different strings are derived, the grammar is said to be unambiguous.

Simplification of grammars:

Removing all unnecessary symbols.

Step:

1. Removal of useless symbols (Reduction of CFG)
2. Removal of null symbols
3. Removal of ~~em~~ unit symbols.

Reduction of CFG - CFG are reduced in two phases

Phase 1: Derivation of equivalent grammar G' from CFG, G , such that each variable derives terminal string.

Procedure:

- Step 1: Include all symbols w_i that derives some terminals and initialize $i=1$.
- Step 2: Include symbols w_{i+1} , that derives w_i .
- Step 3: Increment i & repeat step 2 until $w_{i+1} = w_i$.
- Step 4: Include all production rules that have w_i in it.

Phase 2: Derivation of equivalent grammar G'' from CFG, G' , such that each symbols appears in form (i.e. derives from starting symbol)

Procedures:

- Step 1: Include start symbol in y_i and initialize $i=1$.
- Step 2: Include symbols y_{i+1} , that can be derived from y_i .
- Step 3: Increment i and repeat step 2 until $y_{i+1} = y_i$.
- Step 4: Include all production rules that have y_i in it.

eg- Find RG equivalent to G showing p.r as,
 $S \rightarrow AC|B, A \rightarrow a, C \rightarrow c|BC, E \rightarrow e|aA$.

Phase 1:

$\Rightarrow$ Here, $T = \{a, c, e\}$

$$W_1 = \{A, C, E\}$$

$$W_2 = \{A, C, E, S\} \quad W_3 = \{A, C, E, S\}$$

$\therefore G' = \{\{A, C, E, S\}, \{a, c, e\}, P, S\}$ where

(*) $\rightarrow P: \{S \rightarrow AC, A \rightarrow a, C \rightarrow c, E \rightarrow aA|e\}$

Phase 2:

$$Y_1 = \{S\}$$

$$Y_2 = \{AC, S\}$$

$$Y_3 = \{A, C, S, a, c\}$$

$$Y_4 = \{S, A, C, a, c\}$$

$G'' = \{(S, A, C, a, c), \{a, c\}, P, S\}$ where

$P: S \rightarrow AC, A \rightarrow a, C \rightarrow c$.

This is the reduced grammar.

2. Find the reduced grammar eq. to G having p.r as $S \rightarrow ABa|BC, A \rightarrow aC|BCC, C \rightarrow a, B \rightarrow bcc, D \rightarrow E, E \rightarrow d, F \rightarrow e$

$\Rightarrow$ Phase 1: $T = \{a, b, c, d, e\}$

$$W_1 = \{S, A, B, C, D, E, F\}$$

$$W_2 = \{S, A, B, C, D, E, F\} \quad W_3 = \{S, A, B, C, D, E, F\}$$

$\therefore G' = \{(S, A, B, C, D, E, F), \{a, b, c, d, e\}, P, S\}$ where

$P: \{S \rightarrow ABa|BC, A \rightarrow aC|BCC, C \rightarrow a, B \rightarrow bcc, D \rightarrow E, E \rightarrow d, F \rightarrow e\}$

Phase 2: $Y_1 = \{S\}$

$$Y_2 = \{A, B, C, a\}$$

$$Y_3 = \{S, A, B, C, a, b, c\}$$

$$Y_4 = \{S, A, B, C, a, b, c\}$$

$\therefore G'' = \{(S, A, B, C), \{a, b, c\}, P, S\}$ where

$P: \{S \rightarrow ABa|BC, A \rightarrow aC|BCC, B \rightarrow bcc, C \rightarrow a\}$

This is the reduced grammar.

A → ε null

Removal of unit symbol - Any production rule of the form $A \rightarrow B$ where $A, B \in \text{non-terminal symbols}$ are unit symbols.

Step 1: To remove $A \rightarrow B$, add production $A \rightarrow \alpha$ whenever $B \rightarrow \alpha$ occurs in the grammar, where $\alpha \in \text{terminal or null}$.

Step 2: Remove $A \rightarrow B$ from grammar.

Step 3: Repeat steps ^{1 & 2} until all unit production are removed.

Q.1 Remove unit production from grammar whose production rule is given by $P: S \rightarrow XY, X \rightarrow a, Y \rightarrow Z|b, Z \rightarrow M, M \rightarrow N, N \rightarrow a$.

solⁿ Here, unit production are: $Y \rightarrow Z, Z \rightarrow M, M \rightarrow N$

1) Since we have $N \rightarrow a$, we add $M \rightarrow a$.

$\therefore P: S \rightarrow XY, X \rightarrow a, Y \rightarrow Z|b, Z \rightarrow M, M \rightarrow a, N \rightarrow a$

2) Since we have $M \rightarrow a, Z \rightarrow a$.

$\therefore P: S \rightarrow XY, X \rightarrow a, Y \rightarrow Z|b, Z \rightarrow a, M \rightarrow a, N \rightarrow a$.

3) Since we have $Z \rightarrow a, Y \rightarrow a|b$.

$\therefore P: S \rightarrow XY, X \rightarrow a, Y \rightarrow a|b, Z \rightarrow a, M \rightarrow a, N \rightarrow a$.

Also, remove unreachable symbols.

$\therefore$ The required production rule is

$P: S \rightarrow XY, X \rightarrow a, Y \rightarrow a|b$.

2. Remove unit production from grammar whose production rule is given by $S \rightarrow A|bb, A \rightarrow B|b, B \rightarrow S$.

solⁿ Here, unit production are: $S \rightarrow A, A \rightarrow B, B \rightarrow S$.

1) Since we have $\overset{S \rightarrow bb}{B \rightarrow a}$, we add $B \rightarrow bb$

$\therefore P: S \rightarrow A|bb, A \rightarrow B|b, B \rightarrow b|a$

2) Since we have $B \rightarrow b|a$, we add $A \rightarrow a|bb$

$\therefore P: S \rightarrow A|bb, A \rightarrow b|b|a, B \rightarrow b|a$.

27. Since we have $A \rightarrow a|b|bb$, we add $S \rightarrow a|b|bb$.
 $\therefore P: S \rightarrow a|b|bb$, which is the required p.r.
 $A \rightarrow a|b|bb$
 $B \rightarrow b|ba$

3. $S \rightarrow Aa|B$, $B \rightarrow A|bb$, $A \rightarrow a|bc|B$.

Here, unit productions are:

$S \rightarrow B$, $B \rightarrow A$, $A \rightarrow B$

1. Since $B \rightarrow A$, we add $B \rightarrow a|bc|bb$.
 $\therefore P: S \rightarrow Aa|B$, $B \rightarrow A|bb$, $A \rightarrow a|bc|bb$.

2. Since $A \rightarrow a|bc|bb$, we add $B \rightarrow a|bb|a|bc$.
 $\therefore P: S \rightarrow Aa|B$, $B \rightarrow a|bb|a|bc$, $A \rightarrow a|bc|bb$.

3. Since $A \rightarrow a|bc|bb$, $B \rightarrow a|bb|a|bc$, we add $S \rightarrow a|bb|a|bc$.
 $\therefore P: S \rightarrow Aa|bb|a|bc$, $B \rightarrow a|bb|a|bc$, $A \rightarrow a|bc|bb$.

Also, remove non-reachable symbols.

$\therefore P: S \rightarrow Aa|bb|a|bc$, $B \rightarrow a|bb|a|bc$.

Removal of null symbol:

In a CFG, a null terminal symbol A is nullable variable or null production if there is a production, $A \rightarrow \epsilon$ ($A \rightarrow \Lambda$) or there is a derivation that starts at A and leads to ϵ or Λ . i.e. $A \xrightarrow{*} \Lambda/\epsilon$.

Procedure:

Step 1: To remove $A \rightarrow \epsilon$, look for productions whose right side contain A .

Step 2: Replace each occurrence of A in each of these production with ϵ .

Step 3: Add the resultant production to the grammar.

1. Remove null production from following grammar:
 $S \rightarrow ABAC, A \rightarrow aA/\epsilon, B \rightarrow bB/\epsilon, C \rightarrow c.$

Here, null productions are: $A \rightarrow \epsilon, B \rightarrow \epsilon.$

1. To eliminate $A \rightarrow \epsilon$:

a)	$S \rightarrow ABAC (A \rightarrow \epsilon)$	b)	$S \rightarrow ABAC$	c)	$S \rightarrow ABAC$
	$\rightarrow ABEC$		$\rightarrow EBAC$		$\rightarrow EBEC$
	$\rightarrow ABC$ ($A \rightarrow \epsilon$)		$\rightarrow BAC$		$\rightarrow BC$

So, we get $S \rightarrow ABC/BAC/BC$

$\therefore P: S \rightarrow AB$

d) $A \rightarrow aA[A \rightarrow \epsilon]$

$\rightarrow a \quad ABAC$

$\therefore P: S \rightarrow ABC/BAC/BC, A \rightarrow a|B \rightarrow aA, B \rightarrow bB/\epsilon, C \rightarrow c.$

2. To eliminate $B \rightarrow \epsilon$:

a)	$S \rightarrow ABAC [A \rightarrow \epsilon B \rightarrow \epsilon B \rightarrow \epsilon]$	b)	$S \rightarrow ABC$	c)	$S \rightarrow BAC$
	$\rightarrow AAC$		$\rightarrow AC$		$\rightarrow AC$

d) $S \rightarrow BC$

$\rightarrow C$. So, we get, $S \rightarrow AAC/AC/C$

e) $B \rightarrow bB$

$\rightarrow b. \therefore B \rightarrow b.$

$\therefore$ The new production rule is: $S \rightarrow AAC/AC/C/ABAC/ABC/BAC/BC$
 $A \rightarrow a|aA, B \rightarrow bB/b, C \rightarrow c$

2. $S \rightarrow aA, A \rightarrow BB, B \rightarrow aBb/\epsilon$

Here, null production: $B \rightarrow \epsilon.$

- 1) To remove $B \rightarrow \epsilon$:

a)	$A \rightarrow BB \{$	b)	$A \rightarrow BB$	c)	$A \rightarrow BB$
	$\rightarrow B$		$\rightarrow B$		$\rightarrow \epsilon$

d) $B \rightarrow aBb$

$\rightarrow ab$

$\therefore P: S \rightarrow aA, A \rightarrow B/\epsilon, B \rightarrow aBb/ab$

2. To remove $A \rightarrow \epsilon$:

a) $S \rightarrow aA (A \rightarrow \epsilon)$
 $\rightarrow a$

$\therefore p: S \rightarrow aA|a, A \rightarrow BB|B, B \rightarrow aB|ab.$

3. Simplify the following grammar:

$S \rightarrow aA|aBB, A \rightarrow aAA|E, B \rightarrow bB|bbc, C \rightarrow B.$

$\Rightarrow$ Here, it is better to remove null symbol first & remove unit prodⁿ, then finally remove useless symbols.

① Removing E prodⁿ gives the resulting grammar as:
 $S \xrightarrow{aA} a|aBB, A \rightarrow aA|aAA|a, B \rightarrow bB|bbc, C \rightarrow B.$

② Removing unit prodⁿ gives the resulting grammar:

$S \rightarrow aA|a|aBB$

$A \rightarrow aA|aAA|a$

$B \rightarrow bB|bbc$

$C \rightarrow bB|bbc$

③ Removing useless symbols:

Here, B and C are useless symbols (B and C in RHS can't be replaced by terminals)

$\therefore S \rightarrow aA|a, A \rightarrow aA|aAA|a.$

4. Remove null production from following grammar:

$S \rightarrow ABaC, A \rightarrow BC, B \rightarrow b|E, D \rightarrow d, C \rightarrow D|E.$

~~①~~

$S \rightarrow ABa|A \cancel{C}|Aa, A \rightarrow B|C|E, B \rightarrow b, D \rightarrow d, C \rightarrow D$

$S \rightarrow Ba| \cancel{A}C|a, A \rightarrow B|C, B \rightarrow b, D \rightarrow d, C \rightarrow D$

2. $S \rightarrow aBC|a$, $A \rightarrow b|d$
 $B \rightarrow b$, $D \rightarrow d$, $C \rightarrow d$

3. $\therefore$ The required grammar is $P: S \rightarrow aC|Bda$, $A \rightarrow b|d$, $B \rightarrow b$, $D \rightarrow d$, $C \rightarrow d$

Here, null production are: $B \rightarrow \epsilon$, $C \rightarrow \epsilon$.

1. To eliminate $B \rightarrow \epsilon$:

i) $S \rightarrow ABaC$ ii) $A \rightarrow BC$
 $\rightarrow AaC$ $\rightarrow C$

$\therefore P: S \rightarrow ABaC|AaC$, $A \rightarrow C|BC$, $B \rightarrow b$, $D \rightarrow d$, $C \rightarrow D|\epsilon$

2. To eliminate $C \rightarrow \epsilon$:

i) $S \rightarrow ABaC$ ii) $S \rightarrow AaC$ iii) $S \rightarrow AaC$
 $\rightarrow ABa$ $\rightarrow Aa$ $\rightarrow Aa$

iv) So, we get $S \rightarrow ABaC|AaC|ABa|Aa$

v) $A \rightarrow C$ vi) $A \rightarrow BC$ $\rightarrow \epsilon$ $\rightarrow B$

So, we get $A \rightarrow C|BC|B|\epsilon$

$\therefore P: S \rightarrow ABaC|AaC|ABa|Aa|ABa|Aa$, $A \rightarrow C|BC|B|\epsilon$, $B \rightarrow b$, $D \rightarrow d$, $C \rightarrow D$.

3. To eliminate $A \rightarrow \epsilon$:

i) $S \rightarrow ABaC$ ii) $S \rightarrow AaC$ iii) $S \rightarrow ABa$ iv) $S \rightarrow Aa$
 $\rightarrow BaC$ $\rightarrow aC$ $\rightarrow Ba$ $\rightarrow a$

v) $S \rightarrow ABa$ vi) $S \rightarrow Aa$ $\rightarrow A$
 $\rightarrow Ba$ $\rightarrow a$

So, we get, $S \rightarrow ABaC|AaC|ABa|Aa|ABa|Aa|BaC|aC|Ba$.

The required production rule is

$P: S \rightarrow ABaC|AaC|ABa|Aa|ABa|Aa|BaC|aC|Ba$,
 $A \rightarrow C|BC|B$, $B \rightarrow b$, $D \rightarrow d$, $C \rightarrow D$.

Normal forms -

1. Chomsky Normal form (CNF) - A CFG is said to be in CNF if the p.r are in the following form -

$A \rightarrow a$, $A \rightarrow BC$, where, A, B, C are non-terminals and a is terminal.

Procedure: Ⓟ

Step 1: If the start symbol S occurs on right side^{of starting symbol}, create a new start symbol S' . And a prodⁿ $S' \rightarrow S$.

Step 2: Remove null production.

Step 3: Remove unit production.

Step 4: Change string of terminals on RHS of prodⁿ if it exceeds as follows:

Suppose we have production,

$$S \rightarrow a_1 a_2 a_3 \dots a_n$$

Then introduce non-terminal symbol C_i for a_i as:

$$C_{a_1} \rightarrow a_1 \quad C_{a_2} \rightarrow a_2 \quad C_{a_3} \rightarrow a_3$$

Step 5: To restrict no. of variables on RHS, introduce new variable and separate them as follows:

Suppose we have 'n' non-terminals productions as shown below with '5' non-terminals:

$$Y \rightarrow X_1 X_2 X_3 X_4 X_5$$

Add $n-2$ new production with $n-2$ new non-terminals and modify productions as:

$$R_1 \rightarrow X_4 X_5$$

$$R_2 \rightarrow X_3 R_1$$

$$R_3 \rightarrow X_2 R_2$$

$$Y R_4 \rightarrow X_1 R_3$$

Step 6: If the right side of any production is in the form $A \rightarrow aB$, then the production is replaced by $A \rightarrow XB, X \rightarrow a$.

1. Convert the following CFG into CNF:

$S \rightarrow ASA|aB, A \rightarrow B|S, B \rightarrow b|E$.

→ ① Since S is on RHS, we create S' and $S' \rightarrow S$ is added to the production.

P: $S' \rightarrow S, S \rightarrow ASA|aB, A \rightarrow B|S, B \rightarrow b|E$

② Remove null production: $B \rightarrow E$

P: $S' \rightarrow S, S \rightarrow ASA|aB|a, A \rightarrow B|S|E, B \rightarrow b$

To eliminate $A \rightarrow E$:

P: $S' \rightarrow S, S \rightarrow ASA|aB|a|AS|SA, A \rightarrow B|S, B \rightarrow b$

③ Remove unit production: $S' \rightarrow S, S \rightarrow S, A \rightarrow B, A \rightarrow S$

After removing $S \rightarrow S$,

P: $S' \rightarrow S, S \rightarrow ASA|aB|a|AS|SA, A \rightarrow B|S, B \rightarrow b$

④ After removing $S' \rightarrow S$,

P: $S' \rightarrow ASA|aB|a|AS|SA, S \rightarrow ASA$

After removing $A \rightarrow B$,

P: $S' \rightarrow ASA|aB|a|AS|SA, S \rightarrow ASA|aB|a|AS|SA, A \rightarrow b|S, B \rightarrow b$

After removing $A \rightarrow S$,

P: $S' \rightarrow ASA|aB|a|AS|SA, S \rightarrow ASA|aB|a|AS|SA$

$A \rightarrow b|ASA|aB|a|AS|SA, B \rightarrow b$.

④ Now, we find prodⁿ having more than two variable on RHS,

$S' \rightarrow ASA, S \rightarrow ASA, A \rightarrow ASA$

① $S' \rightarrow ASA$

$\rightarrow AX$ where $[X \rightarrow SA]$

② $A \rightarrow ASA$

$\rightarrow AX$

③ $S \rightarrow ASA$

$\rightarrow AX$

After removing, we get,

$$B: S' \rightarrow AX|aB|a|AS|SA, S \rightarrow AX|aB|a|AS|SA$$

$$A \rightarrow AX|b|aB|a|AS|SA, B \rightarrow b, X \rightarrow SA$$

Now, change

- ⑤ Since the prodⁿ $S' \rightarrow aB, A \rightarrow aB, A \rightarrow aB$, we get,
 $S' \rightarrow YB, A \rightarrow YB, A \rightarrow YB$ where $[Y \rightarrow a]$

After removing, we get,

$$S' \rightarrow AX|YB|a|AS|SA$$

$$S \rightarrow AX|YB|a|AS|SA$$

$$A \rightarrow AX|b|YB|a|AS|SA, B \rightarrow b, X \rightarrow SA, Y \rightarrow a.$$

This is the required CNF.

2. $S \rightarrow bA|aB, A \rightarrow bAA|aS|a, B \rightarrow aBB|bS|b$

$\Rightarrow$

Greibach Normal Form (GNF) -

A CFG is in GNF if the productions are in the following form: $A \rightarrow b$, $A \rightarrow bC_1C_2C_3 \dots C_n$, where $C_1, C_2, \dots, C_n$ are non-terminals and b is terminal.

Procedure:

Step 1: Check if the given CFG has any unit production or null production and remove if there are any.

Step 2: Check whether the CFG is already in CNF and convert it to CNF if it's not.

Step 3: ^{Change} Since the name of non-terminal symbol into some A_i in ascending order of i .

eg - $S \rightarrow CA|BB$
 $B \rightarrow b|SB$
 $C \rightarrow b, A \rightarrow a$

Here, we replace S with A_1 , C with A_2 , A with A_3 , B with A_4 .

$\therefore A_1 \rightarrow A_2A_3|A_4A_4$

$A_4 \rightarrow b|A_1A_4$

$A_2 \rightarrow b, A_3 \rightarrow a$.

Step 4: Alter the rules so that non-terminals are in ascending order such that if the production is of the form $A_i \rightarrow A_jX$, then $i < j$ & should never be $i \geq j$.

In the above example in step 3, consider,

$A_4 \rightarrow b|A_1A_4$ ($\because i > j$ i.e. $4 > 1$)

$A_4 \rightarrow b|A_2A_3A_4|A_4A_4A_4$ (Replacing $A_1 \rightarrow A_2A_3|A_4A_4$)

$A_4 \rightarrow b|\underbrace{bA_3A_4}_{\text{GNF}}|A_4A_4A_4$ ($A_2 \rightarrow b$)

GNF

Here, $A_1 \rightarrow A_1 A_1 A_1$ is left recursion.

Step 5: Remove left recursion.

Introduce a new variable to remove the left recursion.

in GNF? No
x?

$$A_1 \rightarrow b \mid b A_3 A_1 \mid A_1 A_1 A_1$$

$$Z \rightarrow A_1 A_1 \mid A_1 A_1 Z$$

$$A_1 \rightarrow b Z \mid b A_3 A_1 Z \mid b \mid b A_3 A_1$$

The new grammar is.

$$A_1 \rightarrow A_2 A_3 \mid A_1 A_1$$

$$A_1 \rightarrow b \mid b A_3 A_1 \mid b Z \mid b A_3 A_1 Z$$

$$A_3 \rightarrow a, A_2 \rightarrow b$$

$$Z \rightarrow A_1 A_1 \mid A_1 A_1 Z$$

After replacing, we get,

$$A_1 \rightarrow b A_3 \mid b A_1 \mid b A_3 A_1 A_1 \mid b Z A_1 \mid b A_3 A_1 Z A_1$$

$$Z \rightarrow b A_1 \mid b A_3 A_1 Z A_1 \mid b Z A_1 \mid b A_3 A_1 A_1 \mid b A_1 Z \mid b A_3 A_1 A_1 Z \mid b Z A_1 A_1 \mid b A_3 A_1 Z A_1 Z$$

$$A_1 \rightarrow b \mid b A_3 A_1 \mid b Z \mid b A_3 A_1 Z$$

$$A_3 \rightarrow a, A_2 \rightarrow b$$

$\therefore$ This is the required GNF.

Q.1. Convert the CFG to GNF :

$$S \rightarrow A A \mid a, A \rightarrow S S \mid b.$$

solⁿ Replacing S with A_1 , A with A_2 .

$$A_1 \rightarrow A_2 A_2 \mid a$$

$$A_2 \rightarrow A_1 A_1 \mid b$$

Consider $A_2 \rightarrow A_1 A_1 \mid b$ and after replacing,

$$A_2 \rightarrow A_2 A_2 A_1 \mid a A_1 \mid b$$

Consider $A_2 \rightarrow A_2 A_2 A_1$: We introduce a new var. Z .

$$Z \rightarrow A_2 A_1 \mid A_2 A_1 Z$$

$\therefore$ The new grammar is:

$$A_1 \rightarrow A_2 A_2 \mid a$$

$$Z \rightarrow A_2 A_1 \mid A_2 A_1 Z$$

$$A_2 \rightarrow a A_1 \mid b \mid A_1 Z \mid b Z$$

After replacing, we get,

$$A_1 \rightarrow a A_1 A_2 \mid b A_2 \mid a A_1 Z A_2 \mid b Z A_2 \mid a$$

$$Z \rightarrow a A_1 A_1 \mid b A_1 A_1 \mid a A_1 Z A_1 \mid b Z A_1 \mid a A_1 A_1 Z \mid b A_1 Z \mid a A_1 Z A_1$$

$$A_2 \rightarrow a A_1 \mid b \mid a A_1 Z \mid b Z \quad | b Z A_1 Z$$

This is the required GNF.

Finite automata cannot accept L i.e. the string of the form $a^n b^n$ as it has to remember the strings before it goes to the next part of the string and compare it with the first half but doesn't have the capability to remember anything.

This problem can be solved by adding an auxiliary memory in the form of stack, which leads to the generation of a pushdown automata.

A PDA is a finite automaton with control of both an input tape and stack to store what it has read. It consists of an input tape, finite control, and stack as shown below:

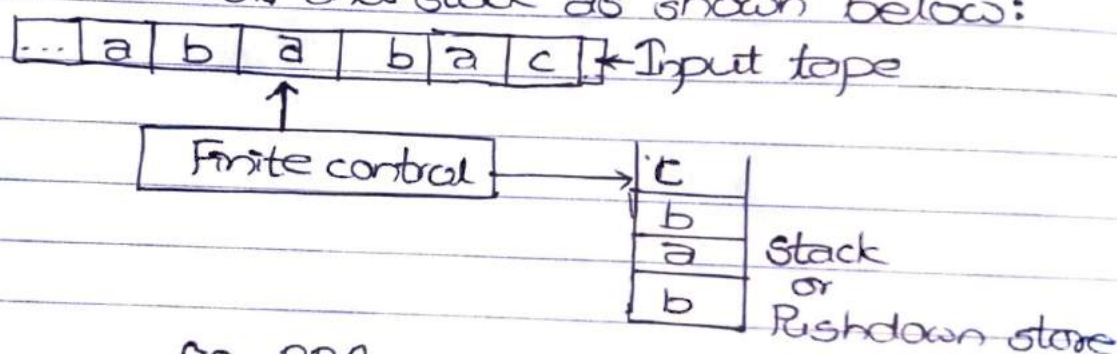


fig- PDA

A pushdown automata is a system which is defined by seven tuples $M = \{Q, \Sigma, \Gamma, \delta, q_0, Z_0, F\}$ where, $Q =$ ^{finite} set of states

$\Sigma =$ input alphabets

$\Gamma =$ elements of stack

$\delta =$ transition function $[\delta(q, a, X)] \rightarrow (p, \frac{d}{r}, t)$

$q_0 =$ initial state

$Z_0 =$ top of stack

$F =$ set of final states.

top of stack
i/p. symbol
n.s.



1. $\delta(q, a, x) \rightarrow (p, y)$ means whenever PDA is in state q with x , the top of stack, may read 'a' from the input tape, replace 'x' by 'y' on the top of stack and enter state p .
2. $\delta(q, a, \wedge) \rightarrow (p, y)$ means y is pushed at the top of stack.
3. $\delta(q, a, y) \rightarrow (p, \wedge)$ means y is popped from the top of stack.

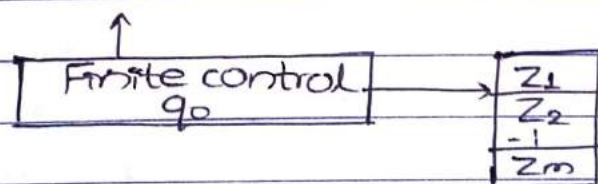
Instantaneous Description (ID): ID describes is an informal notation of how a PDA computes the input string and makes decision that a string is accepted or rejected.

(q, x, α) is an ID.

where q = set of states, $x = \Sigma^*$, $\alpha = \Gamma$

$(q, a_1 a_2 \dots a_n, z_1 z_2 \dots z_m)$

$[a_1 | a_2 | \dots | a_n]$



eg- $\delta(q_0, bacab, z_0)$

$\delta(q_0, bacab, z_0) \vdash \delta(q_0, acab, bz_0)$

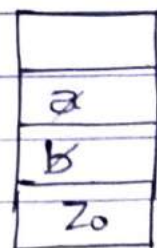
$\vdash \delta(q_0, cab, abz_0)$

$\vdash \delta(q_0, ab, abz_0)$

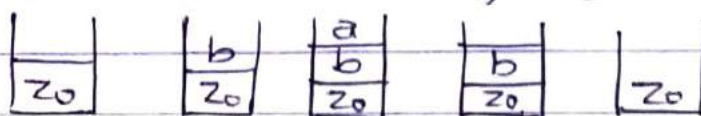
$\vdash \delta(q_1, b, bz_0)$

$\vdash \delta(q_1, \epsilon, z_0)$

$\vdash \delta(q_f, z_0)$



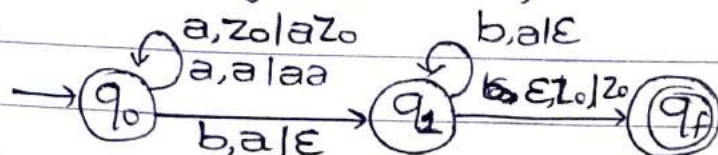
(final content of PDS)



(c is ignored)

1. Design a PDA for $L = \{a^n b^n, n \geq 1\}$

- i. First push 'a' into the stack.
- ii. Reading each 'b', remove each 'a' from stack.
- iii. If all a's and b's are removed & stack is empty, then string L is accepted.



$\therefore$ PDA is defined as $M = \{Q, \Sigma, \Gamma, \delta, q_0, Z_0, F\}$ where,
 $Q = \{q_0, q_1, q_f\}$ $\Gamma =$ elements of stack
 $\Sigma = \{a, b\}$ $q_0 = \{q_0\}$ $F = \{q_f\}$
 $Z_0 =$ top of stack

$\therefore \delta$ is defined as:

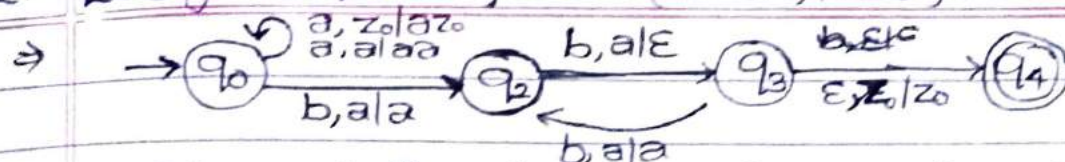
$$\begin{aligned} \delta(q_0, a, z_0) &= (q_0, a z_0) \\ \delta(q_0, a, a) &= (q_0, a a) \\ \delta(q_0, b, a) &= (q_1, \epsilon) \\ \delta(q_1, b, a) &= (q_1, \epsilon) \\ \delta(q_1, \epsilon, z_0) &= (q_f, z_0) \end{aligned}$$

Let us consider string 'aaabbbb'.

$$\begin{aligned} \delta(q_0, aaabbbb, z_0) &\vdash \delta(q_0, aabbbb, a z_0) \\ &\vdash \delta(q_0, abbbb, a a z_0) \\ &\vdash \delta(q_0, bbbb, a a a z_0) \\ &\vdash \delta(q_1, bbb, a a a z_0) \\ &\vdash \delta(q_1, bb, a a z_0) \\ &\vdash \delta(q_1, b, a z_0) \\ &\vdash \delta(q_1, \epsilon, z_0) \\ &\vdash \delta(q_f, z_0) \end{aligned}$$

$\therefore$ The string is accepted.

2. Design a PDA for $L = \{a^n b^{2n}, n \geq 1\}$.



∴ PDA is defined as $M = \{Q, \Sigma, \Gamma, \delta, q_0, Z_0, F\}$ where

$Q = \{q_0, q_1, q_2, q_3, q_4\}$

$\Gamma = \text{stack elements}$

$\Sigma = \{a, b\}$

$q_0 = \{q_0\}$. $F = \{q_4\}$

∴ δ is defined as: $\delta(q_0, a, z_0) = (q_0, az_0)$

$\delta(q_0, a, az_0) = (q_0, aaz_0)$

$\delta(q_0, b, a) = (q_1, a)$

$\delta(q_1, b, a) = (q_2, a)$

$\delta(q_2, b, a) = (q_3, \epsilon)$

$\delta(q_3, b, a) = (q_2, a)$

$\delta(q_3, \epsilon, z_0) = (q_4, z_0)$

Let us consider string 'aabbabb'.

$\delta(q_0, aabbabb, z_0) \vdash \delta(q_0, abbbabb, az_0)$

$\vdash \delta(q_0, bbbabb, aaz_0)$

$\vdash \delta(q_1, bbbabb, aaz_0)$

$\vdash \delta(q_2, bbbabb, aaz_0)$

$\vdash \delta(q_3, bbbabb, aaz_0)$

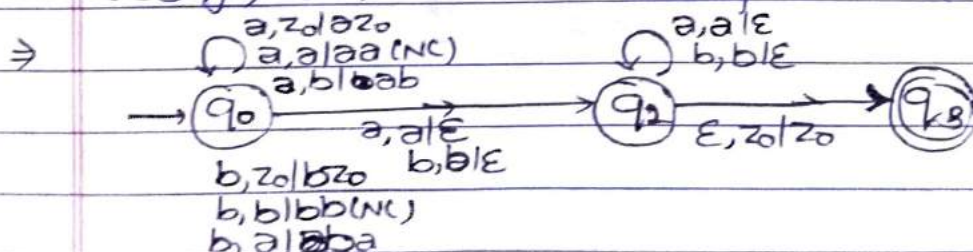
$\vdash \delta(q_2, bbbabb, aaz_0)$

$\vdash \delta(q_3, \epsilon, z_0)$

$\vdash \delta(q_4, z_0)$.

∴ The string is accepted.

3. Design a PDA that accepts $L = \{ww^R, w \in \{a, b\}^*\}$



Let the required PDA be $M = \{Q, \Sigma, \Gamma, \delta, q_0, Z_0, F\}$

where $Q = \{q_0, q_1, q_2, q_3\}$

$\Sigma = \{a, b\}$

$Z_0 = \text{top of stack}$

$q_0 = \{q_0\}$

$F = \{q_3\}$

$\therefore \delta$ is defined as: $\delta(q_0, a, z_0) = \delta(q_0, az_0)$

$$\delta(q_0, a, a) = \delta(q_0, aa) \quad (NC)$$

$$\delta(q_0, a, b) = \delta(q_0, ab)$$

$$\delta(q_0, b, z_0) = \delta(q_0, bz_0)$$

$$(NC) \quad \delta(q_0, b, b) = (q_0, bb), \quad \delta(q_0, b, a) = (q_0, ba)$$

$$(c) \quad \delta(q_0, a, a) = (q_1, \epsilon), \quad \delta(q_0, b, b) = (q_1, \epsilon) \quad (c)$$

$$\delta(q_1, a, a) = (q_2, \epsilon), \quad \delta(q_1, b, b) = (q_2, \epsilon)$$

$$\delta(q_2, \epsilon, z_0) = (q_2, z_0)$$

Let us consider the string 'abbaabba'.

$$\delta(q_0, abbaabba, z_0) \vdash \delta(q_0, bbaabba, az_0)$$

$$\vdash \delta(q_0, baabba, bazo)$$

$$\vdash \delta(q_0, aabba, bbazo)$$

$$\vdash \delta(q_0, abba, abbaazo)$$

$$\vdash \delta(q_1, bba, bbazo)$$

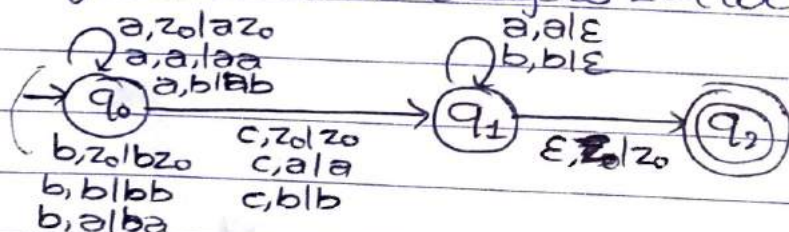
$$\vdash \delta(q_1, ba, bazo) \vdash \delta(q_1, a, az_0)$$

$$\vdash \delta(q_1, \epsilon, z_0)$$

$$\vdash \delta(q_2, z_0)$$

4. Design PDA that accepts $L = \{w c w^R : w \in \{a, b\}^*\}$.

$\Rightarrow$



$\therefore M = \{Q, \Sigma, \Gamma, \delta, q_0, Z_0, F\}$, where

$Q = \{q_0, q_1, q_2\}$, $\Sigma = \{a, b, c\}$, $q_0 = \{q_0\}$, $F = \{q_2\}$

δ is defined as:

$$\delta(q_0, a, z_0) = (q_0, az_0)$$

$$\delta(q_0, a, a) = (q_0, aa)$$

$$\delta(q_0, a, b) = (q_0, ab)$$

$$\delta(q_0, b, z_0) = (q_0, bz_0)$$

$$\delta(q_0, b, b) = (q_0, bb)$$

$$\delta(q_0, b, a) = (q_0, ba)$$

$$\delta(q_0, c, z_0) = (q_1, z_0)$$

$$\delta(q_1, a, a) = (q_1, \epsilon)$$

$$\delta(q_0, c, a) = (q_1, a)$$

$$\delta(q_1, b, b) = (q_1, \epsilon)$$

$$\delta(q_0, c, b) = (q_1, b)$$

$$\delta(q_1, \epsilon, z_0) = (q_2, z_0)$$

Let us consider the string 'aabcbaa'.

$$\therefore \delta(q_0, aabcbaa, z_0) \vdash \delta(q_0, abcbbaa, az_0)$$

$$\vdash \delta(q_0, bebaa, aaz_0)$$

$$\vdash \delta(q_0, cbbaa, baaz_0)$$

$$\vdash \delta(q_1, bbaa, baaz_0) \vdash \delta(q_1, aa, aaz_0)$$

$$\vdash \delta(q_1, a, aaz_0) \vdash \delta(q_1, \epsilon, z_0)$$

$$\vdash \delta(q_2, z_0).$$

$\therefore$ The string is accepted.

Equivalence of PDA and CFG :

Construction of PDA for given CFG :-

Example: Given a grammar, $S \rightarrow OBB$, $B \rightarrow OS/1S/O$. Build a PDA.

$\Rightarrow$ Define PDA as $M: (Q, \Sigma, \Gamma, \delta, q_0, z_0, F)$

q_2, δ, q_0, F

δ is given by

$$\delta(q_0, \epsilon, z_0) = (q_0, Sz_0)$$

$$\delta(q_0, \epsilon, S) = (q_1, OBBz)$$

$$\delta(q_1, \epsilon, B) = (q_1, OS), (q_1, 1S), (q_1, O)$$

$$\delta(q_1, O, O) = (q_1, \epsilon)$$

$$\delta(q_1, 1, 1) = (q_1, \epsilon)$$

$$\delta(q_1, \epsilon) = (q_f, z_0)$$

$$\therefore Q = \{q_0, q_1, q_f\}, \Sigma = \{O, 1\}, q_0 = \{q_0\}, z_0$$

$$\Gamma = \{S, z_0, B, O, 1\}, F = \{q_f\}$$

Applying δ for '010000':

$$\delta(q_0, 010000, z_0) \vdash \delta(q_0, 010000, Sz_0)$$

$$\vdash \delta(q_1, 010000, OBBz_0)$$

$$\vdash \delta(q_1, 10000, BBz_0)$$

$$\vdash \delta(q_1, 10000, 1SBz_0)$$

$$\vdash (q_1, 0000, SBz_0)$$

$$\vdash (q_1, 0000, 0BBBz_0)$$

$$\vdash (q_1, 000, BBBz_0)$$

$$\vdash (q_1, 0, 0z_0)$$

$$\checkmark \vdash (q_1, 000, 0BBz_0) \nearrow$$

$$\vdash (q_1, \epsilon, z_0)$$

$$\vdash (q_1, 00, BBz_0)$$

$$\vdash (q_f, z_0)$$

$$\vdash (q_1, 00, 0Bz_0)$$

$$\vdash (q_1, 0, Bz_0)$$

2. $S \rightarrow aSa | bSb | c$. Design PDA.

$$\Rightarrow M = \{Q, \Sigma, \Gamma, \delta, q_0, z_0, F\}$$

$$\delta \text{ is given by } \delta(q_0, \epsilon, z_0) = (q_0, Sz_0)$$

$$\delta(q_0, \epsilon, S) = (q_0, aSa), (q_0, bSb), (q_0, c).$$

$$\delta(q_0, a, a) = (q_0, \epsilon)$$

$$\delta(q_0, b, b) = (q_0, \epsilon)$$

$$\delta(q_0, a, a) = (q_1, \epsilon), \delta(q_0, b, b) = (q_1, \epsilon)$$

$$\delta(q_0, c, c) = (q_1, \epsilon)$$

$$\delta(q_1, \epsilon, z_0) = (q_f, z_0z_0)$$

Applying δ for 'abcba':

$$\delta(q_0, abcba, z_0) \vdash \delta(q_0, abcba, Sz_0)$$

$$\vdash \delta(q_0, abcba, aSaz_0)$$

$$\vdash \delta(q_0, bcba, Saaz_0)$$

$$\vdash \delta(q_0, bcba, bSbaz_0)$$

$$\vdash \delta(q_0, cba, Sbaza_0)$$

$$\vdash \delta(q_0, cba, cbaza_0)$$

$$\vdash \delta(q_1, ba, baza_0)$$

$$\vdash \delta(q_1, a, aza_0)$$

$$\vdash \delta(q_1, \epsilon, za_0)$$

$$\vdash \delta(q_f, \epsilon, za_0)$$

000000

000000

Construct CFG from PDA:

$$PDA = \{Q, \Sigma, \Gamma, \delta, q_0, z_0, F\}$$

$$CFG = \{V, \Sigma, P, S\}$$

Here, V can be defined as: $V = \{Q \Gamma Q\}$

eg: $Q = \{q_0, q_1\}, \Gamma = \{a\}$

$$\therefore V = \{[q_0 a q_1], [q_0 a q_0], [q_1 a q_1], [q_1 a q_0]\}$$

1. $\delta(q_0, a, z_0) = (q_0, \epsilon)$

$$P: [q_0 z_0 q_0] \rightarrow a$$

2. $\delta(q_0, a, z_0) = (q_0, a z_0)$

$$[q_0 z_0] \rightarrow a [q_0 a] \rightarrow [z_0] [z_0]$$

3. Suppose if $\delta(q_0, a, z_0) = (q, \epsilon x x)$.

$$[q_0 z_0] \Rightarrow a [q a] \Rightarrow [x] \rightarrow [x]$$

same same

4. $\delta(q_0, b, a) = (q_1, a)$

$$[q_0 a q_0] \Rightarrow b [q_1 a q_0]$$

$$[q_0 a q_1] \Rightarrow b [q_1 a q_1]$$

Q. Given $PDA = \{ (q_0, q_1), (a, b), (a, z_0), \delta, q_0, z_0, \phi \}$ &

δ is given by: $\delta(q_0, a, z_0) = (q_0, a z_0)$

$$\delta(q_0, a, a) = (q_0, a a)$$

$$\delta(q_0, b, a) = (q_1, \epsilon)$$

$$\delta(q_1, b, a) = (q_1, \epsilon)$$

$$\delta(q_1, \epsilon, z_0) = (q_1, \epsilon)$$

$\Rightarrow$ For $\delta(q_0, a, z_0) = (q_0, a z_0)$

~ P1: $[q_0 z_0 q_0] \Rightarrow a [q_0 a q_0] \Rightarrow [q_0 z_0 q_0]$

P2: $[q_0 z_0 q_1] \Rightarrow a [q_0 a q_1] \Rightarrow [q_1 z_0 q_1]$

~ P3: $[q_0 z_0 q_1] \rightarrow a [q_0 a q_0] \Rightarrow [q_0 z_0 q_0]$

~ P4: $[q_0 z_0 q_0] \rightarrow a [q_0 a q_1] \Rightarrow [q_1 z_0 q_0]$

For $\delta(q_0, a, a) = (q_0, \epsilon a)$:

- $P_5: [q_0 a q_0] \Rightarrow \epsilon [q_0 a q_0] \Rightarrow [q_0 a q_0]$
 $P_6: [q_0 a q_0] \rightarrow a [q_0 a q_1] \Rightarrow [q_0 a q_0]$
 $P_7: [q_0 a q_1] \rightarrow a [q_0 a q_0] \Rightarrow [q_0 a q_1]$
 $P_8: [q_0 a q_1] \rightarrow a [q_0 a q_1] \Rightarrow [q_1 a q_1]$

For $\delta(q_0, b, a) = (q_1, \epsilon)$:

- $P_9: [q_0 a q_0] \rightarrow b [q_1]$
 $P_{10}: [q_0 a q_1] \rightarrow b$

For $\delta(q_1, b, a) = (q_1, \epsilon)$:

- $P_{11}: [q_1 a q_0] \rightarrow b$ $P_{12}: [q_1 a q_1] \rightarrow b$

For $\delta(q_1, \epsilon, z_0) = (q_1, \epsilon)$:

- $P_{13}: [q_1 z_0 q_0] \rightarrow \epsilon$ $P_{14}: [q_1 z_0 q_1] \rightarrow \epsilon$

$P_{12}: \therefore S \rightarrow [q_0 \overset{\text{top of stack}}{\underset{\text{initial state}}{z_0}} q_0] \mid [q_0 z_0 \underset{\text{comb. of states}}{q_1}]$

(left hand side
symbols
never used
in)

Now, we also reduce useless symbols (non-generating or non-reachable symbols).

Removing P_5 as $[q_0 a q_0]$ is not generating any symbol. P_1, P_2, P_4, P_7

Remove P_5 as it's generating itself.

Remove $[q_0 z_0 q_0]$ from S so we have:

$S \rightarrow [q_0 z_0 q_1]$

$\therefore$ PR of CFG can be written as:

$G = \{V, \Sigma, P, S\}$

$\Sigma = \{a, b\}$, $S = \{S\}$

$V = \{[q_0 z_0 q_1], [q_0 a q_1], [q_0 a q_1], [q_1 z_0 q_1], S\}$

$\therefore P.R$ is defined as: $S \rightarrow [q_0 q_0 q_1]$

$P_2: [q_0 z_0 q_1] \rightarrow a [q_0 a q_1] [q_1 z_0 q_1]$

$[q_0 a q_1] \rightarrow a [q_0 a q_1] [q_1 a q_1]$

$[q_0 a q_1] \rightarrow b$

$[q_1 a q_1] \rightarrow b$ $[q_1 z_0 q_1] \rightarrow \epsilon$.

Pumping lemma for CFG - Let L be a CFL and n be the length of the string or pumping length such that:

- i) Every $z \in L$ with $|z| \geq n$ can be written as $uvwxy$ for some u, v, w, x, y .
- ii) length of $|vx| \geq 1$ or $|vx| > 0$.
- iii) length $|vwx| \leq n$ (i.e if $v, u = \Lambda$, $|vwx| = n$)
- iv) $uv^kwx^ky \in L$ for all $k \geq 0$. (i.e generating infinite no. of strings for all values of k)